Generating All Triangulations of Biconnected Plane Graphs in Amortized Constant Time Per Triangulation

Tawkir Aziz Rahman
Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology (BUET)
Dhaka-1000, Bangladesh

June 2, 2026

Abstract

We present an algorithm for generating all triangulations of a biconnected plane graph G with n vertices in $O(1)$ amortized time per triangulation and $O(n)$ space. Our algorithm establishes a double-layer tree structure consisting of a recursive tree of faces and, for each face, a local genealogical tree of triangulations. The key innovation is a novel "safe root vertex" selection strategy that prevents the multi-edge problem inherent in biconnected graphs, thereby extending the optimal complexity previously achievable only for triconnected graphs. We prove that every face contains at least two safe root vertices regardless of prior triangulations, and we show how to find one in $O(n)$ time using a two-pointer technique. A dynamic global chord set together with a valid generating set (VGS) allows constant-time edge flips and safe pruning of invalid branches. Extensive experiments on 35 diverse test cases validate the theoretical bounds: the algorithm generates tens of millions of triangulations while maintaining nanosecond-level per-triangulation time and linear memory usage. To the best of our knowledge, this is the first algorithm for generating all triangulations of a biconnected plane graph with $O(1)$ amortized time per triangulation.

Acknowledgements

I express my sincere gratitude to my supervisor for his invaluable guidance and encouragement throughout this research. I also thank the authors of the foundational work on triconnected plane graph triangulations, Mohammad Tanvir Parvez, Md. Saidur Rahman, and Shin-ichi Nakano, whose elegant algorithm provided the starting point for this investigation. Finally, I am grateful to the Department of Computer Science and Engineering, BUET, for providing the necessary resources and a conducive research environment.

Contents

List of Figures

List of Algorithms

Chapter 1

Introduction

1.1 Motivation and Applications

The problem of generating all triangulations of plane graphs arises in many areas of computational geometry [2], VLSI floorplanning [4], and graph drawing [7]. A triangulation of a plane graph is a maximal set of non-crossing chords that partition each face into triangles. Enumerating all possible triangulations is essential for exploring geometric configurations, optimizing layouts, and testing hypotheses about graph properties.

In VLSI design, for instance, different triangulations of a floorplan correspond to different routing possibilities. In mesh generation, enumerating triangulations helps to find optimal finite element meshes. In graph drawing, triangulations are used to create straight-line embeddings and to study graph properties.

1.2 Challenges in Generating All Triangulations

The number of triangulations of a planar graph can be exponential in the number of vertices. For a convex polygon with n vertices, the number of triangulations is the Catalan number C_{n-2}, which grows as $\Theta(4^n n^{-3/2})$. For general plane graphs, the count can be even larger. Therefore, any algorithm that lists all triangulations must handle an exponential output size.

The main challenges in designing such algorithms are threefold:

1. **Exponential output:** The algorithm must run in time proportional to the number of triangulations; otherwise it would be inefficient.

2. **Avoiding duplicates:** Since the output is huge, storing previously generated triangulations to check for duplicates would require exponential space and time.

3. **Space efficiency:** The algorithm should use only polynomial space (preferably linear in the input size) and not store all outputs simultaneously.

Classical approaches to generating combinatorial objects include orderly algorithms [6] and reverse search [1]. Orderly algorithms generate objects in a canonical form and output only those that are canonical, thus avoiding duplicates without storing all previous objects. Reverse search defines a spanning tree on the graph of objects and traverses it using a parent function, again without storing the entire set.

1.3 Previous Work

There is a rich literature on triangulation enumeration. For convex polygons, it is well known that triangulations correspond to binary trees, and efficient generation algorithms exist that run in constant amortized time per triangulation [3].

For planar graphs, the problem is more challenging. Li and Nakano [5] gave an algorithm to generate all biconnected plane triangulations with at most n vertices in $O(r^2n)$ time per triangulation, where r is the number of vertices on the outer face. Later, Nakano improved this to $O(rn)$ time per triangulation.

The most relevant prior work is the algorithm by Parvez, Rahman, and Nakano [8], which generates all triangulations of a *triconnected* plane graph in $O(1)$ amortized time per triangulation using $O(n)$ space. This algorithm defines a genealogical tree of triangulations where each node is a triangulation and each edge corresponds to a chord flip. By traversing this tree in a depth-first manner, all triangulations are generated without duplicates.

However, the algorithm of Parvez et al. is limited to triconnected graphs. As we will explain in Section 1.5, extending it directly to biconnected graphs leads to the multi-edge problem caused by separation pairs. This limitation is the main motivation for our work.

1.4 The Parvez–Rahman–Nakano 2011 Algorithm (Overview)

We briefly recall the key ideas of the algorithm for triconnected plane graphs, as they form the foundation of our approach.

Given a triconnected plane graph G with n vertices, the algorithm processes all faces simultaneously. For each face F_i (a cycle), it constructs a genealogical tree of triangulations. The root of the tree is a *fan triangulation* obtained by picking an arbitrary vertex as the root and connecting it to all non-neighbouring vertices on the face. From the root, child triangulations are generated by flipping a *generating chord* (a chord that, when flipped, yields a new triangulation whose leftmost blocking chord is the flipped chord). The parent of any non-root triangulation is obtained by flipping its leftmost blocking chord. This parent–child relationship is unique, and the tree contains every triangulation exactly once.

The algorithm runs in $O(1)$ amortized time per triangulation and uses $O(n)$ space. The key properties that enable this efficiency are:

- Flipping a chord takes constant time.
- The depth of the genealogical tree is $O(n)$.
- The number of generating chords decreases along any path from the root.

1.5 Limitation of Prior Work: The Multi-Edge Problem in Biconnected Graphs

The Parvez–Rahman–Nakano algorithm works only for triconnected plane graphs. The reason is that in triconnected graphs, no separation pair exists, so flipping chords never creates multi-edges. In biconnected graphs, however, separation pairs are possible, and independent triangulation of different faces can lead to the same chord being added twice.

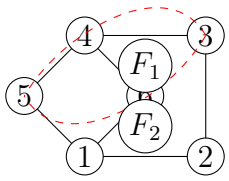


Figure 1.1: A biconnected plane graph with a separation pair $(3, 5)$. Triangulating faces F_1 and F_2 independently can add the chord $(3, 5)$ twice, creating a multi-edge.

Figure 1.1 illustrates a biconnected graph where vertices 3 and 5 form a separation pair: removing them disconnects the graph. If we triangulate the outer face F_1 and the inner face F_2 independently using the algorithm of Parvez et al., we might add the chord $(3, 5)$ in both faces, leading to a multi-edge. Such multi-edges are not allowed in simple graphs, and the algorithm would generate invalid triangulations.

Thus, a direct application of the Parvez–Rahman–Nakano algorithm to biconnected graphs fails. Our goal is to overcome this limitation while preserving the optimal $O(1)$ amortized time per triangulation.

1.6 Our Contribution

We present the first algorithm for generating all triangulations of a *biconnected* plane graph in $O(1)$ amortized time per triangulation and $O(n)$ space. Our main innovations are:

1. **Local genealogical trees:** Instead of processing all faces simultaneously, we process faces one by one in a fixed order. For each face, we construct a local genealogical tree of triangulations that are compatible with previously triangulated faces.

2. **Safe root vertex selection:** We introduce the concept of a *safe root vertex* for a face. A safe root vertex guarantees that the fan triangulation from that vertex does not create any multi-edge with chords from previous faces. We prove that every face in a biconnected plane graph contains at least two safe root vertices, regardless of the chords already present.

3. **Linear-time safe root finding:** Using a two-pointer technique based on planarity, we find a safe root vertex in $O(n)$ time, where n is the number of vertices in the face. This ensures that the building cost per face can be amortized.

4. **Valid generating set (VGS):** We maintain a set of generating chords whose flip will not create a multi-edge. When flipping a chord, only four border chords change their opposite pairs; we update the VGS in constant time using linked lists.

5. **Global chord set with dynamic push/pop:** A global hash set stores all chords currently on the exploration path. This allows $O(1)$ conflict detection and supports backtracking in the recursive DFS.

6. **Pruning strategy:** If a blocking chord (i.e., a chord that does not have the root vertex as an endpoint) conflicts with a chord from a previous face, all descendant triangulations are invalid. We prune such branches safely, without losing any valid triangulation.

Our algorithm achieves the same optimal complexity as the triconnected case while handling the strictly larger class of biconnected graphs.

1.7 Comparison with Existing Algorithms

Table 1.1 summarises the differences between our algorithm and previous work.

Table 1.1: Comparison of triangulation enumeration algorithms.

Algorithm	**Graph Class**	**Time per triangulation**	**Space**
Reverse search [1]	General	$O(n^2)$ or worse	$O(n)$
Li–Nakano [5]	Biconnected	$O(r^2 n)$	—
Nakano (improved)	Biconnected	$O(rn)$	—
Parvez et al. [8]	Triconnected	$O(1)$ amortized	$O(n)$
This work	**Biconnected**	$\mathbf{O(1)}$ **amortized**	$\mathbf{O(n)}$

Our algorithm is the first to achieve constant amortised time for the class of biconnected plane graphs, which includes many practical instances.

1.8 Thesis Organization

The remainder of this thesis is structured as follows.

- **Chapter 2** introduces the necessary graph-theoretic concepts: connectivity, planarity, cycles, chords, triangulations, flipping operations, and genealogical trees. Special emphasis is given to separation pairs and the distinction between triconnected and biconnected graphs.
- **Chapter 3** describes the baseline algorithm for generating all triangulations of a single cycle (the Parvez–Rahman–Nakano algorithm) in detail. We present the definitions of blocking chords, generating chords, and leftmost blocking chords, and prove uniqueness.
- **Chapter 4** clarifies the distinction between our naming of the generating set and that of the original paper, and proves the condition for flipping generating chords.
- **Chapter 5** explains why the original algorithm fails for biconnected graphs due to separation pairs and the multi-edge problem.
- **Chapter 6** introduces our pruning strategy based on the persistence of chords that do not involve the root vertex.
- **Chapter 7** presents the multi-layer DFS that processes faces one by one, maintaining a global chord set and backtracking.
- **Chapter 8** addresses the problem of generating chords themselves causing multi-edges and introduces the concept of safe root vertices.
- **Chapter 9** proves a lower bound of $\Omega(n)$ triangulations per face, which is essential for amortised analysis.
- **Chapter 10** gives an $O(n)$ two-pointer algorithm for finding a safe root vertex.
- **Chapter 11** describes the valid generating set (VGS) and how to update it in constant time during flips.
- **Chapter 12** presents the complete algorithm in pseudocode and proves its correctness.
- **Chapter 13** analyses the time and space complexity, proving $O(1)$ amortised time per triangulation and $O(n)$ space.
- **Chapter 14** provides experimental validation on 35 test cases, including cycles, grids, and complex graphs, confirming the theoretical bounds.

- **Chapter 15** discusses why the algorithm cannot be extended to 1-connected graphs in general, giving examples where safe roots do not exist.
- **Chapter 16** outlines future work, including extensions to 1-connected graphs and 1-planar graphs.
- **Chapter 17** concludes the thesis with a summary of contributions and final remarks.

Chapter 2

Preliminaries

In this chapter, we define the basic concepts used throughout the thesis. We assume the reader is familiar with elementary graph theory, but we provide precise definitions to avoid ambiguity. Special emphasis is given to notions that are essential for biconnected plane graphs, such as separation pairs and separation vertices, which do not appear in the triconnected case.

2.1 Basic Graph Definitions

Let $G = (V, E)$ be a connected simple graph with vertex set V and edge set E. Each edge $e \in E$ is an unordered pair $\{u, v\}$ with $u, v \in V$ and $u \neq v$. We denote an edge between u and v by (u, v) or (v, u).

The **degree** of a vertex v is the number of edges incident to v. A graph is **simple** if it has no loops or multiple edges.

The **connectivity** $\kappa(G)$ of a graph G is the minimum number of vertices whose removal disconnects the graph (or reduces it to a single vertex). A graph is k**-connected** if $\kappa(G) \geq k$. Equivalently, removing fewer than k vertices leaves the graph connected.

In particular:

- A graph is **2-connected** (or **biconnected**) if it has no articulation vertices (cut vertices) and remains connected after the removal of any single vertex.
- A graph is **3-connected** (or **triconnected**) if it remains connected after the removal of any two vertices.

Definition 2.1 (Separation pair)**.** In a biconnected graph, a pair of vertices $\{a, b\}$ is called a **separation pair** if removing a and b disconnects the graph into at least two components. A biconnected graph that contains a separation pair is not triconnected.

Definition 2.2 (Separation vertex)**.** In a 1-connected graph, a vertex v is a **separation vertex** (or **cut vertex**) if removing v disconnects the graph. A graph is biconnected if and only if it has no separation vertices.

The distinction between biconnected and triconnected graphs is crucial: triconnected graphs have no separation pairs, while biconnected graphs may have them. The presence of separation pairs is the main obstacle when extending triangulation generation algorithms from triconnected to biconnected graphs.

2.2 Planar and Plane Graphs

A graph G is **planar** if it can be drawn in the plane without any edge crossings. Such a drawing is called an **embedding**. A **plane graph** is a planar graph together with a fixed embedding in the plane.

A plane graph divides the plane into connected regions called **faces**. The unbounded region is the **outer face**; all other faces are **inner faces**. Each face is bounded by a closed walk along the edges of the graph.

If every face boundary contains exactly three edges, the plane graph is called a **plane triangulation**. In this thesis, edges are not required to be straight lines; triangulation is understood combinatorially.

2.3 Separation Pairs and Separation Vertices

We now discuss separation pairs in more detail, as they are central to the biconnected case.

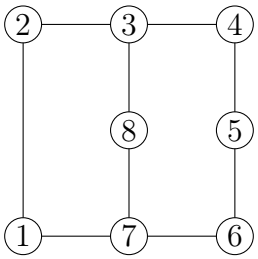


Figure 2.1: A biconnected plane graph. Vertices 3 and 7 form a separation pair.

Figure 2.1 shows a biconnected plane graph with eight vertices. Removing vertices 3 and 7 disconnects the remaining vertices into three components: $\{1, 2\}$, $\{4, 5, 6\}$, and $\{8\}$. Therefore, $(3, 7)$ is a separation pair.

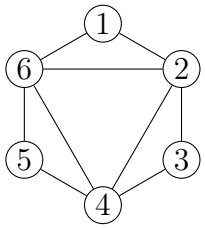
Remark 2.1. In a biconnected plane graph, a separation pair (a, b) implies that the graph can be split into two subgraphs that share only a and b. Consequently, there exist two

faces (one on each side of the pair) that can potentially contain the same chord (a, b), leading to a multi-edge if both faces are triangulated independently.

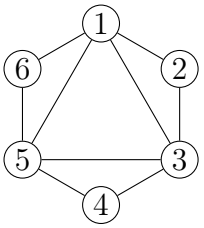
2.4 Faces, Cycles, and Chords

A **cycle** C in a graph is a sequence of distinct vertices $v_1, v_2, \ldots, v_k$ such that $(v_i, v_{i+1}) \in E$ for $1 \leq i < k$ and $(v_k, v_1) \in E$. In a plane graph, each face corresponds to a cycle that bounds it (taking the outer face as a cycle as well).

Let $C = \langle v_1, v_2, \ldots, v_n \rangle$ be a cycle, listed in counterclockwise order. An edge (v_i, v_j) is called a **chord** of C if it is not an edge of C and its interior lies inside the face bounded by C. A chord (v_i, v_j) divides the cycle into two smaller cycles: $v_i, v_{i+1}, \ldots, v_j$ and $v_j, v_{j+1}, \ldots, v_i$.



(a) A triangulation of a hexagon.



(b) Another triangulation of the same hexagon.

Figure 2.2: Two different triangulations of a cycle with six vertices.

2.5 Triangulation of a Cycle

A **triangulation** of a cycle C is a maximal set of non-intersecting chords that partition the interior of C into triangles. The sides of the triangles are either chords or edges of C. Figure 2.2 shows two triangulations of a hexagon.

Equivalently, a triangulation of a cycle with n vertices contains exactly $n - 3$ chords. This follows from Euler's formula applied to the planar graph formed by the cycle and its chords.

2.6 Labeled vs. Unlabeled Triangulations

If the vertices of the cycle are numbered sequentially from v_1 to v_n, then a triangulation is called a **labeled triangulation**. The order of vertices matters: two triangulations that differ only by a rotation or reflection are considered distinct if the labels are different.

If the vertices are not numbered, the triangulations are called **unlabeled**. In this case, rotationally equivalent or mirror-image triangulations are considered the same. In

this thesis, we work exclusively with **labeled triangulations** unless stated otherwise.

2.7 Flipping Operation

A fundamental operation for transforming one triangulation into another is the **flip**. Consider two adjacent triangles that share a chord and together form a quadrilateral. Removing the common chord and adding the opposite diagonal produces a new triangulation.

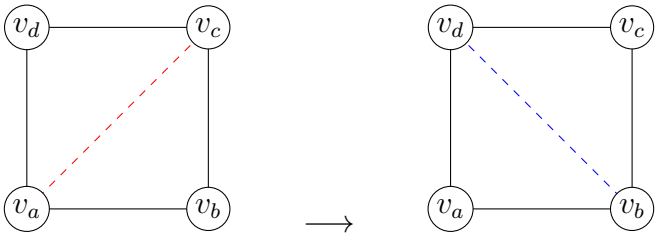


Figure 2.3: Flipping the diagonal (v_a, v_c) to (v_b, v_d) in a quadrilateral.

Formally, let (v_i, v_j) be a chord that belongs to two triangles (v_i, v_j, v_p) and (v_i, v_j, v_q) on opposite sides. These four vertices form a quadrilateral v_p, v_i, v_q, v_j in clockwise order. Flipping (v_i, v_j) means removing it and adding the opposite chord (v_p, v_q). The resulting graph is again a triangulation of the same cycle.

2.8 Visibility, Blocking Chords, and Generating Chords

Fix a cycle $C = \langle v_1, v_2, \ldots, v_n \rangle$ and choose a vertex, say v_1, as the **root vertex**. In a triangulation T of C, we say that a vertex v_j is **visible** from v_1 if the chord (v_1, v_j) belongs to T.

In the root triangulation (the fan from v_1), every vertex except the neighbours v_2 and v_n is visible from v_1. In other triangulations, some vertices may be hidden behind **blocking chords**.

Definition 2.3 (Blocking chord)**.** A chord (v_i, v_j) of a triangulation T is called a **blocking chord** if:

1. Neither v_i nor v_j is the root vertex v_1, and
2. The chord (v_i, v_j) is visible from v_1 in the sense that there is a triangle (v_1, v_i, v_j) in T.

Equivalently, a blocking chord is a chord that does not have v_1 as an endpoint but is directly connected to v_1 via triangles.

Blocking chords prevent some vertices from being visible from the root. Flipping a blocking chord replaces it with a chord that is incident to the root, thereby increasing the number of visible vertices.

Definition 2.4 (Generating chord)**.** In a triangulation T of a cycle with root vertex v_1, a chord (v_1, v_j) is called a **generating chord** if flipping it produces a new triangulation T' whose leftmost blocking chord is exactly the newly added chord.

The precise condition for a chord to be generating involves the leftmost blocking chord, which we define next.

2.9 Leftmost Blocking Chord

Among the blocking chords of a triangulation, we single out the one that is "leftmost" with respect to the cyclic order around the root.

Definition 2.5 (Leftmost blocking chord)**.** Let T be a triangulation of a cycle $C = \langle v_1, v_2, \ldots, v_n \rangle$ with root vertex v_1. Let $(v_b, v_{b'})$ be a blocking chord of T with $b < b'$. It is the **leftmost blocking chord** if no other blocking chord (v_i, v_j) satisfies $j < b'$ (i.e., its larger index is smaller than b'). In other words, among all blocking chords, the one with the smallest value of the larger endpoint is the leftmost.

Remark 2.2. The term "leftmost" comes from drawing the cycle with v_1 at the top and the remaining vertices arranged clockwise around it. Blocking chords then appear as chords that "block" the view to vertices on the left side.

We now prove that the leftmost blocking chord is unique. The following argument is taken from the author's original writing, polished for clarity.

Lemma 2.1 (Uniqueness of the leftmost blocking chord)**.** *In any triangulation T of a cycle $C = \langle v_1, v_2, \ldots, v_n \rangle$ that is not the root triangulation, there is exactly one leftmost blocking chord.*

Proof. Suppose, for contradiction, that there exist two distinct blocking chords that both qualify as leftmost. Let the two chords be $(v_b, v_{b'})$ and $(v_c, v_{c'})$ with $b < b'$ and $c < c'$. By definition of leftmost, we must have $b' = c'$ (otherwise the one with the smaller larger endpoint would be unique). Hence both chords share the same larger endpoint, say $v_{b'} = v_{c'} = v_k$.

Since both chords are blocking, each forms a triangle with the root vertex v_1. Thus we have triangles (v_1, v_b, v_k) and (v_1, v_c, v_k). The vertices v_b and v_c lie on the same side of the chord (v_1, v_k). Without loss of generality, assume $b < c$.

Consider the quadrilateral formed by v_1, v_b, v_k, v_c. Because the triangulation is planar and all faces are triangles, the chord (v_b, v_k) and (v_c, v_k) cannot both be present without

crossing. In fact, only one of them can be a chord; the other must be replaced by the opposite diagonal of the quadrilateral. However, if both are present, they would create a face with four vertices, contradicting that the triangulation is maximal. Therefore, it is impossible to have two distinct blocking chords with the same larger endpoint. Hence the leftmost blocking chord is unique. □

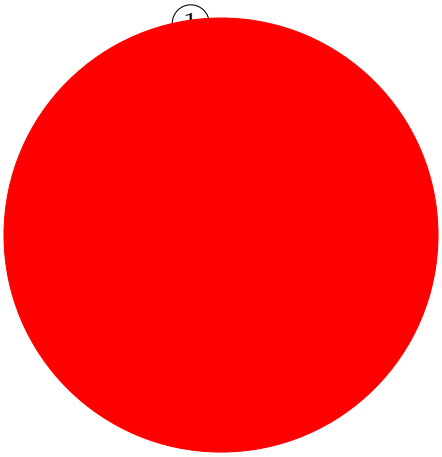


Figure 2.4: Illustration of the leftmost blocking chord. Here (v_4, v_6) is the leftmost blocking chord because no other blocking chord has a larger endpoint smaller than 6.

2.10 Genealogical Tree of Triangulations

The parent–child relationship defined by flipping the leftmost blocking chord yields a rooted tree structure on the set of all triangulations of a cycle.

Definition 2.6 (Genealogical tree)**.** Let C be a cycle with a fixed root vertex v_1. The **genealogical tree** $\mathcal{T}(C)$ of triangulations of C is defined as follows:

- The root is the fan triangulation where all chords are incident to v_1.
- For any non-root triangulation T, its parent $P(T)$ is obtained by flipping the leftmost blocking chord of T.
- Conversely, a triangulation T is a child of T' if $T' = P(T)$.

The genealogical tree has the following properties (proved in Chapter 3):

- Every triangulation appears exactly once.
- The root has $n - 3$ children (all chords are generating).
- The depth of the tree is at most $n - 2$.
- Flipping a generating chord takes constant time.

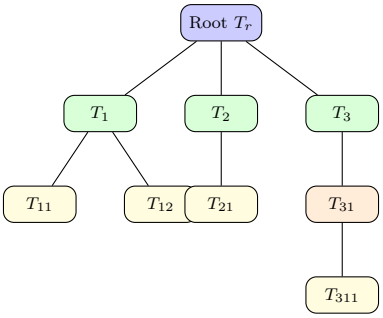


Figure 2.5: A schematic genealogical tree for a cycle. Each node represents a triangulation, and edges correspond to flipping a generating chord.

This tree structure is the basis of the algorithm for generating all triangulations of a cycle, which we recall in Chapter 3.

2.11 Summary

We have introduced the essential graph-theoretic concepts: connectivity, planarity, faces, cycles, chords, triangulations, and the flipping operation. We defined blocking chords, generating chords, and the leftmost blocking chord, and proved its uniqueness. The genealogical tree provides a canonical parent–child relationship among triangulations of a cycle.

For biconnected graphs, we highlighted the significance of separation pairs, which are the main obstacle to extending the triconnected algorithm. These concepts will be used throughout the remainder of the thesis.

Chapter 3

Generating All Triangulations of a Cycle

In this chapter we present the algorithm for generating all triangulations of a cycle with labeled vertices. This algorithm, due to Parvez, Rahman, and Nakano [8], forms the core building block of our method for biconnected graphs. We describe it in detail, including the necessary definitions, the parent-child relationship, the data structures, and the complexity analysis.

3.1 Problem Definition

Let $C = \langle v_1, v_2, \ldots, v_n \rangle$ be a cycle with n vertices listed in counterclockwise order. All vertices are labeled, and the embedding is fixed. A **triangulation** of C is a maximal set of non-crossing chords that partition the interior of C into triangles. Any triangulation contains exactly $n - 3$ chords.

We wish to generate *all* triangulations of C without duplication, using $O(1)$ amortized time per triangulation and $O(n)$ space.

3.2 Root Vertex and Root Triangulation

We fix an arbitrary vertex as the **root vertex**. In this chapter we take v_1 as the root without loss of generality. The **root triangulation** T_r is the fan triangulation obtained by connecting v_1 to every other vertex except its neighbours v_2 and v_n (see Figure 3.1). Thus T_r contains the chords $(v_1, v_3), (v_1, v_4), \ldots, (v_1, v_{n-1})$.

In the root triangulation, every vertex except v_2 and v_n is directly visible from v_1 via a chord. We say that v_1 has **full vision** in T_r.

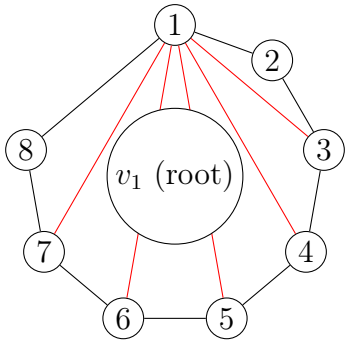


Figure 3.1: Root triangulation (fan) of an octagon. All chords are incident to the root vertex v_1.

3.3 Visibility and Blocking Chords

In a general triangulation T of C, some vertices may be hidden from v_1 by chords that do not involve v_1. These chords are called **blocking chords**.

Definition 3.1 (Visible vertex)**.** A vertex v_j is **visible** from the root v_1 in a triangulation T if the chord (v_1, v_j) belongs to T.

Definition 3.2 (Blocking chord)**.** A chord (v_i, v_j) with $i, j \neq 1$ is called a **blocking chord** if the triangle (v_1, v_i, v_j) is present in T. In other words, both v_i and v_j are visible from v_1, but the chord (v_i, v_j) blocks the view to vertices between v_i and v_j on the cycle.

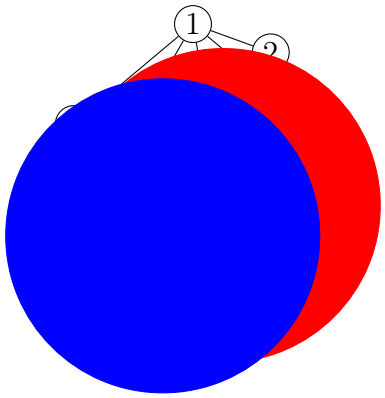


Figure 3.2: Blocking chords in a triangulation. The chords (v_3, v_5) and (v_5, v_7) block the view from v_1 to vertices v_4 and v_6 respectively.

Observe that in the root triangulation there are no blocking chords. As we move away from the root, blocking chords appear and reduce the number of vertices visible from v_1.

3.4 The Leftmost Blocking Chord

Among all blocking chords of a triangulation T, we select a canonical one to define the parent.

Definition 3.3 (Leftmost blocking chord)**.** Let $(v_b, v_{b'})$ with $b < b'$ be a blocking chord of T. It is the **leftmost blocking chord** if no other blocking chord (v_i, v_j) satisfies $j < b'$. In other words, it has the smallest possible larger endpoint.

Lemma 3.1 (Uniqueness)**.** *Any triangulation T that is not the root has exactly one leftmost blocking chord.*

The proof was given in Chapter 2 (Lemma 2.1).

3.5 Parent-Child Relationship

The genealogical tree $\mathcal{T}(C)$ is defined by the following parent function.

Definition 3.4 (Parent of a triangulation)**.** For a non-root triangulation T, let $(v_b, v_{b'})$ be its leftmost blocking chord. The **parent** $P(T)$ is obtained by flipping $(v_b, v_{b'})$ in the quadrilateral that contains it.

Flipping a blocking chord removes it and adds a chord incident to the root, thereby increasing the number of visible vertices. Hence the parent has a clearer view from v_1 than the child.

Lemma 3.2. *Every triangulation $T \neq T_r$ has a unique parent, and the parent has exactly one more chord incident to v_1 than T.*

Proof. Uniqueness follows from the uniqueness of the leftmost blocking chord. Flipping $(v_b, v_{b'})$ replaces it with a chord (v_1, v_k) for some k (the opposite vertex of the quadrilateral). Thus the number of chords incident to v_1 increases by one. □

Conversely, a triangulation T can have several children. Each child is obtained by flipping a **generating chord** of T.

3.6 Generating Chords

Let $(v_b, v_{b'})$ be the leftmost blocking chord of T (if T is the root, we set $b = n$ and $b' = n+1$ for convenience). A chord (v_1, v_j) is called a **generating chord** of T if flipping it produces a new triangulation whose leftmost blocking chord is exactly the newly added chord.

Lemma 3.3 (Condition for generating chords)**.** *A chord (v_1, v_j) is generating if and only if $j \geq b'$ (where b' is the larger endpoint of the leftmost blocking chord, or $j \geq 3$ for the root).*

Proof. The original proof from Parvez et al. [8] uses a case analysis. If $j < b'$, flipping (v_1, v_j) produces a new chord (v_p, v_q) with $q \leq b'$, so the leftmost blocking chord of the child is still $(v_b, v_{b'})$ (or another chord with the same larger endpoint), and the flipped chord is not the leftmost. Therefore the parent of the child would not be T, so (v_1, v_j) is not generating.

If $j \geq b'$, flipping (v_1, v_j) creates a chord (v_p, v_q) with $q > b'$. One can verify that this new chord becomes the leftmost blocking chord of the child, and T is its parent. Hence (v_1, v_j) is generating. □

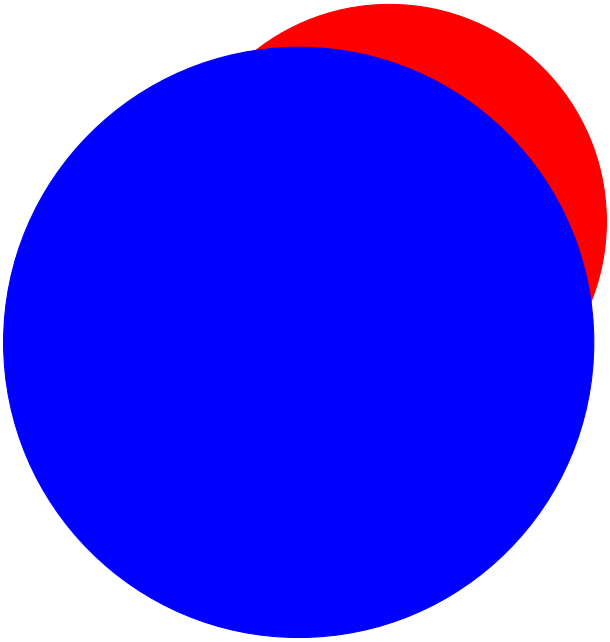


Figure 3.3: Generating chords in a triangulation. The leftmost blocking chord is (v_4, v_6); therefore the generating chords are those with $j \geq 6$: (v_1, v_6), (v_1, v_8), (v_1, v_9).

The root triangulation has no blocking chord; by convention we set $b' = 3$, so all chords (v_1, v_j) with $j = 3, 4, \ldots, n-1$ are generating. Thus the root has $n - 3$ children.

3.7 Data Structures

To represent a triangulation T of C and to generate its children efficiently, we maintain three lists:

- L **(chord list)**: The set of all chords of T, stored in a linked list.
- GS **(generating set)**: The list of generating chords of T. For the root, $GS = L$.
- O **(opposite pairs)**: For each generating chord (v_1, v_j), we store its **opposite pair** $(v_o, v_{o'})$ where $o < j < o'$ and the quadrilateral $(v_1, v_o, v_j, v_{o'})$ is present in T. Flipping (v_1, v_j) replaces it with $(v_o, v_{o'})$.

All three lists require $O(n)$ space because a triangulation of n vertices has exactly $n-3$ chords.

3.8 Algorithm for Generating All Triangulations of a Cycle

We now present the algorithm in pseudocode. The main procedure is a depth-first traversal of the genealogical tree $\mathcal{T}(C)$.

Algorithm 1 GenerateAllTriangulationsCycle

```
1: procedure GenerateAllTriangulationsCycle(C)
2:     v_r ← v_1                                   ▷ Choose root vertex
3:     T_r ← BuildRootTriangulation(C, v_r)
4:     Output(T_r)
5:     DFS(T_r)
6: end procedure
```

Algorithm 2 BuildRootTriangulation

```
1: procedure BuildRootTriangulation(C, v_r)
2:     L ← ∅, GS ← ∅, O ← ∅
3:     for j = 3 to n − 1 do
4:         Add chord (v_r, v_j) to L and GS
5:         Store opposite pair (v_{j−1}, v_{j+1}) for (v_r, v_j) in O
6:     end for
7:     return (L, GS, O)
8: end procedure
```

Algorithm 3 DFS (Depth-first traversal of $\mathcal{T}(C)$)

```
1: procedure DFS(T)
2:     Let (v_b, v_{b'}) be the leftmost blocking chord of T     ▷ If none, set b' = 3
3:     for each generating chord (v_1, v_j) ∈ GS with j ≥ b' do
4:         T' ← Flip(T, (v_1, v_j))
5:         Output(T')
6:         DFS(T')
7:     end for
8: end procedure
```

The flip operation (Algorithm 4) creates a child triangulation in constant time by updating the three lists.

Remark 3.1. Updating the opposite pairs affects at most two chords (the neighbours of the flipped chord). Therefore the entire flip operation takes $O(1)$ time.

Algorithm 4 Flip

1: **procedure** FLIP($T, (v_1, v_j)$)
2: Find opposite pair $(v_o, v_{o'})$ from O for (v_1, v_j)
3: Remove (v_1, v_j) from L and GS
4: Add $(v_o, v_{o'})$ to L (but not to GS)
5: Update opposite pairs of the two neighbouring generating chords (v_1, v_o) and $(v_1, v_{o'})$ (if they exist)
6: Recompute GS for the new triangulation (only chords with $j \geq$ new leftmost blocking chord remain generating)
7: **return** the new triangulation T'
8: **end procedure**

3.9 Correctness

The algorithm correctly generates all triangulations without duplicates because:

1. The parent of any non-root triangulation is uniquely defined (by the leftmost blocking chord). Hence each triangulation appears exactly once in the tree, and the DFS visits each node exactly once.

2. The generating chords are exactly those whose flips produce children that are valid and have the current triangulation as parent (Lemma **??** in the original paper). Thus no child is missed.

3. The recursion terminates because the depth is bounded by the number of chords incident to v_1, which is at most $n - 3$.

3.10 Time and Space Complexity

3.10.1 Time Complexity

Building the root triangulation takes $O(n)$ time (adding $n - 3$ chords and initialising the lists). For each edge flip (generating a child from a parent), we perform a constant number of operations: removing one chord, adding another, and updating at most two opposite pairs. Thus each child is generated in $O(1)$ time.

Let T be the total number of triangulations of C. The number of edges in the genealogical tree is $T - 1$ (one parent edge per non-root node). Since the DFS traverses each edge once, the total number of flip operations is $T - 1$. Hence the total time is $O(n + T) = O(T)$ (because $n = O(T)$ when $n \geq 4$, as Catalan numbers grow exponentially). Therefore the amortised time per triangulation is $O(1)$.

3.10.2 Space Complexity

At any point during the DFS, we store:

- The lists L, GS, O for the current triangulation: $O(n)$.
- The recursion stack: depth at most $n-2$ (each flip increases the number of chords incident to v_1 by one, starting from 0 at the root and reaching at most $n-3$ at leaves). Hence stack size is $O(n)$.

No other storage is required because we do not keep the entire tree in memory. Thus the space complexity is $O(n)$.

3.11 Example: Triangulations of a Hexagon

Figure 3.4 shows the genealogical tree for a hexagon ($n = 6$). The root has $n - 3 = 3$ generating chords, leading to three children. Each child may have fewer generating chords, and the depth is at most 4 (including the root). The total number of triangulations of a convex hexagon is 14, which matches the Catalan number $C_4 = 14$.

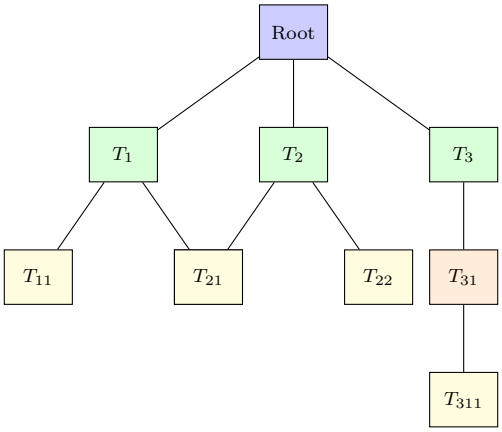


Figure 3.4: Genealogical tree for a hexagon. The root has three children; the total number of nodes is 14.

3.12 Summary

We have described the Parvez–Rahman–Nakano algorithm for generating all triangulations of a labeled cycle. Key properties:

- The genealogical tree is traversed in depth-first order.
- Each child is generated from its parent by flipping a generating chord in $O(1)$ time.
- The space complexity is $O(n)$.

- The amortised time per triangulation is $O(1)$.

This algorithm is used as a subroutine in our method for biconnected plane graphs, where we process each face independently while maintaining a global set of chords to avoid multi-edges.

Chapter 4

Distinction of Our Naming of the Generating Set from the Original Paper

In this chapter we clarify an important difference between the definition of generating chords used in the original paper by Parvez, Rahman, and Nakano [8] and the definition we adopt in this thesis. This distinction is necessary for the introduction of the *valid generating set* (VGS) in later chapters, which allows us to handle multi-edge conflicts in biconnected graphs.

4.1 Original Definition

In the original paper [8], a chord (v_1, v_j) of a triangulation T (with root vertex v_1) is called a **generating chord** if flipping it produces a child triangulation T' whose leftmost blocking chord is exactly the newly added chord. Only chords that satisfy this property are included in the generating set GS. As shown in Chapter 3, this condition is equivalent to requiring that the other endpoint j be at least b', where $(v_b, v_{b'})$ is the leftmost blocking chord of T (or $j \geq 3$ for the root).

Thus, in the original algorithm, GS is a subset of the chords incident to the root, and its size varies with the triangulation.

4.2 Our Definition

In contrast, for the purpose of handling biconnected graphs, we define the **generating set** GS more broadly:

Definition 4.1 (Our generating set)**.** For a triangulation T of a cycle with root vertex v_1, the set GS consists of *all* chords that have v_1 as one endpoint, i.e., all chords of the form (v_1, v_j) for $j = 3, 4, \ldots, n-1$ that are present in T. No further restriction is applied.

In other words, every chord incident to the root is considered a *generating chord* in our terminology, regardless of whether flipping it would yield a child whose leftmost blocking chord is the new chord.

4.3 Why This Distinction Is Necessary

The broader definition is needed for the construction of the **valid generating set** (VGS) introduced in Chapter 11. In the biconnected case, some chords incident to the root, when flipped, may create a multi-edge conflict with chords from previously processed faces. To avoid generating invalid triangulations, we must identify which generating chords are *safe* to flip. The VGS will contain exactly those chords (v_1, v_j) whose flip does not introduce a multi-edge.

If we had used the original, more restrictive definition of generating chords, we would lose the ability to control safety via a simple subset. By including all root-incident chords in GS, we can later filter them based on multi-edge checks, resulting in the VGS. The original condition (whether the flipped chord becomes the leftmost blocking chord) remains relevant for the correctness of the genealogical tree, but it is applied separately when we actually perform the flip.

4.4 Flippability Condition for Our GS

Even though we call all root-incident chords "generating", not every such chord can be flipped while preserving the parent-child relationship defined in Chapter 3. For a chord (v_1, v_j) to be flipped and produce a child triangulation whose parent is the current one, we must still ensure that the flipped chord becomes the leftmost blocking chord of the child. This condition, as proved in the original paper, is $j \geq b'$, where $(v_b, v_{b'})$ is the leftmost blocking chord of the current triangulation (with $b < b'$).

We now restate and prove this condition in a self-contained manner, following the author's original argument.

Lemma 4.1 (Flippability condition)**.** *Let T be a triangulation of a cycle with root vertex v_1. Let $(v_b, v_{b'})$ be the leftmost blocking chord of T (if T is the root, we set $b' = 2$ for convenience). Then flipping a chord (v_1, v_j) in T yields a child triangulation T' for which T is the parent if and only if $j \geq b'$.*

Proof. We consider two cases based on the value of j relative to b'.

Case 1: $j < b'$. Assume $j < b'$. The chord (v_1, v_j) lies entirely to the left of the leftmost blocking chord $(v_b, v_{b'})$ when we walk clockwise from v_1 along the cycle. Flipping (v_1, v_j) replaces it with a chord (v_p, v_q) where both endpoints lie between v_j and the opposite side of the quadrilateral. Because $j < b'$, the new chord (v_p, v_q) will have its larger endpoint at

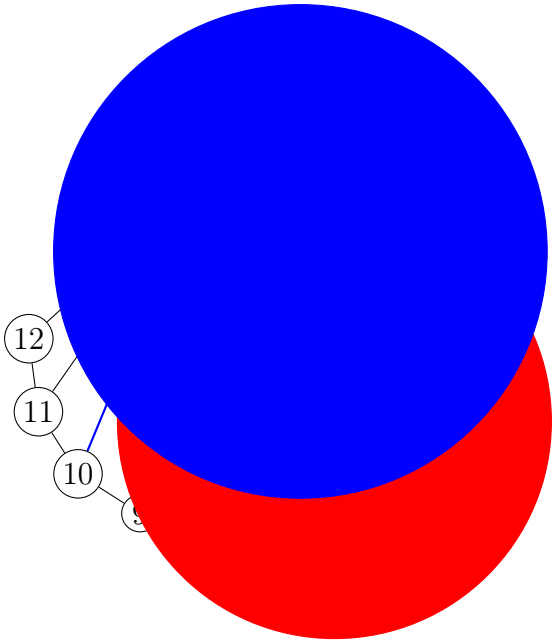


Figure 4.1: Illustration of the leftmost blocking chord $(v_b, v_{b'}) = (v_4, v_7)$ and a generating chord (v_1, v_{10}) with $j = 10 \geq 7$.

most b'. Consequently, the leftmost blocking chord of the resulting triangulation remains $(v_b, v_{b'})$ (or another chord with the same larger endpoint b'). Thus the flipped chord is not the leftmost blocking chord of the child, and the parent of the child would not be T. Hence (v_1, v_j) is not flippable in the sense that it does not preserve the parent-child relationship.

Case 2: $j \geq b'$. Now assume $j \geq b'$. Consider the quadrilateral formed by $v_1, v_b, v_{b'}, v_j$ (or a suitable quadrilateral if $j > b'$). Flipping (v_1, v_j) removes it and adds the opposite diagonal (v_b, v_p) for some p with $p > b'$ (if $j = b'$) or a chord (v_q, v_r) with larger endpoints than b'. In all subcases, the newly added chord has its larger endpoint strictly greater than b'. Therefore it becomes the new leftmost blocking chord of the child triangulation. By definition, T is the parent of this child, so (v_1, v_j) is flippable and preserves the tree structure. □

Remark 4.1. For the root triangulation, there is no blocking chord; we set $b' = 3$ (the smallest index of a chord incident to v_1). Then the condition $j \geq 3$ holds for every chord (v_1, v_j) with $j \geq 3$, so all such chords are flippable, which matches the fact that the root has $n - 3$ children.

4.5 Implication for Our Algorithm

In the original paper, the generating set GS is defined as exactly the set of flippable chords (those with $j \geq b'$). Thus, when the algorithm traverses the genealogical tree, it iterates directly over GS.

In our work, we define GS as *all* chords incident to the root, which is a superset of the flippable chords. When we need to generate children, we must still restrict to those with $j \geq b'$. However, we also need to check an additional condition: whether flipping the chord would create a multi-edge with chords from previously processed faces (in the biconnected setting). Therefore we maintain a separate **valid generating set** (VGS) that contains exactly those chords (v_1, v_j) satisfying:

1. $j \geq b'$ (flippability condition), and
2. The opposite pair of (v_1, v_j) is not already present in the global chord set S (safety condition).

This distinction is crucial for the pruning strategy described in Chapter 6 and for maintaining $O(1)$ amortised time per triangulation.

4.6 Summary

We have clarified the difference between the original definition of generating chords and our broader definition. The broader definition allows us to later filter chords based on multi-edge conflicts, leading to the concept of a valid generating set. The flippability condition $j \geq b'$ remains essential for preserving the parent-child relationship in the genealogical tree. This condition is used in our algorithm when iterating over candidate chords to flip.

Chapter 5

What the Paper Did and Why It Cannot Be Used for Biconnected Graphs

In this chapter we describe the algorithm of Parvez, Rahman, and Nakano [8] for generating all triangulations of a triconnected plane graph. We then explain why this algorithm fails when applied directly to biconnected graphs, focusing on the multi-edge problem caused by separation pairs.

5.1 The Parvez–Rahman–Nakano Algorithm for Triconnected Graphs

The algorithm for triconnected plane graphs follows the same high-level structure as the cycle algorithm presented in Chapter 3, but now multiple faces are processed simultaneously. Given a triconnected plane graph G with n vertices and k faces $F_1, F_2, \ldots, F_k$, the algorithm constructs a *global genealogical tree* $\mathcal{T}(G)$ of triangulations of G.

5.1.1 Building the Root Triangulation

For each face F_i (a cycle), we choose a root vertex arbitrarily and construct the fan triangulation for that face. The root triangulation T_R of G is obtained by taking the union of these face triangulations. Since G is triconnected, the faces are independent: adding chords in one face does not affect the other faces.

The generating set GS^0 of the root triangulation is the union of the generating sets of the individual faces. Since each face with n_i vertices contributes $n_i - 3$ generating chords, the total number of generating chords in the root is $\sum_i (n_i - 3) = O(n)$ (by Euler's formula).

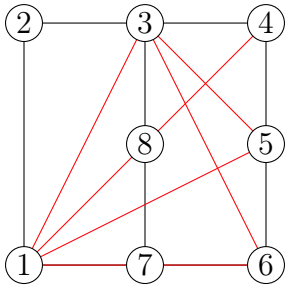


Figure 5.1: A root triangulation of a triconnected plane graph. The root vertex for each face is chosen independently; all chords are added simultaneously.

5.1.2 Generating Child Triangulations

From a triangulation T of G, a child triangulation T' is generated by flipping a generating chord in one eligible face. A face F_i is called **eligible** if all faces with smaller indices have already been processed; the algorithm processes faces in a fixed order.

The key property that enables constant-time generation is that flipping a generating chord in a face affects only that face. Because the graph is triconnected, the flipped chord cannot create a multi-edge with chords from other faces. Therefore, the algorithm can simply flip any generating chord and obtain a new valid triangulation.

5.1.3 Time and Space Complexity

The depth of the global genealogical tree is bounded by the total number of generating chords, which is $O(n)$. Each flip operation takes $O(1)$ time (updating the chord lists and opposite pairs for the affected face). Hence the algorithm generates all triangulations of G in $O(1)$ amortised time per triangulation, using $O(n)$ space.

5.2 Why the Algorithm Cannot Be Used for Biconnected Graphs

The authors of the original paper explicitly limited their algorithm to triconnected plane graphs. The reason is that in biconnected graphs, separation pairs can exist, and these cause a fundamental problem: the **multi-edge problem**.

5.2.1 Separation Pairs in Biconnected Graphs

Recall from Chapter 2 that a separation pair is a pair of vertices whose removal disconnects the graph. Biconnected graphs may contain separation pairs, while triconnected graphs do not.

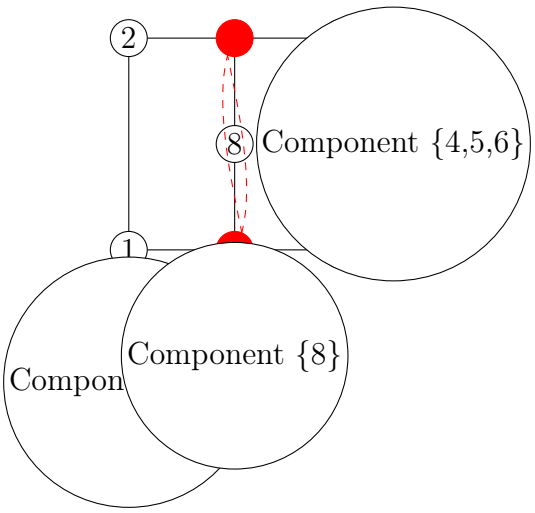


Figure 5.2: A biconnected plane graph with separation pair $(3, 7)$. Removing vertices 3 and 7 leaves three components: $\{1, 2\}$, $\{4, 5, 6\}$, and $\{8\}$.

5.2.2 The Multi-Edge Problem

In a triconnected graph, the triangulation of different faces are independent: any chord added in one face cannot appear in another face because that would require the two faces to share a common chord, which would create a separation pair. In a biconnected graph with a separation pair (a, b), however, there exist two faces (one on each side of the separation pair) that share the same boundary vertices a and b.

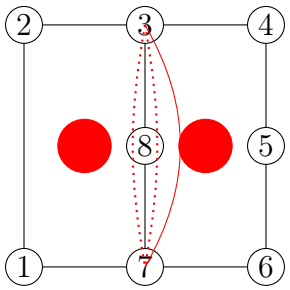


Figure 5.3: The separation pair $(3, 7)$ appears on two faces F_1 and F_2. Triangulating each face independently may add the same chord $(3, 7)$ twice, creating a multi-edge.

If we independently triangulate the two faces using the algorithm of Parvez et al., it is possible that both faces receive the chord (a, b) (the chord that directly connects the separation pair). In a simple graph, having two copies of the same edge is not allowed. Thus the resulting structure would contain a multi-edge, which is not a valid triangulation of a simple graph.

5.2.3 Why the Original Algorithm Fails

The original algorithm was designed assuming that no multi-edge can appear. It processes all faces in parallel, flipping chords without checking whether the new chord already exists in another face. In a triconnected graph this assumption holds, but in a biconnected graph it does not. Therefore a direct application of the algorithm to a biconnected graph would generate many invalid triangulations containing multi-edges.

Moreover, simply discarding invalid triangulations is not sufficient because:

1. Checking for multi-edges would require additional time per flip, breaking the $O(1)$ amortised guarantee.

2. The number of invalid triangulations could be large, wasting substantial computational resources.

3. More importantly, the genealogical tree structure would be corrupted: a child might have a valid parent, but its descendants could all be invalid due to a persistent multi-edge. Pruning such branches requires careful analysis.

5.3 Conclusion

The Parvez-Rahman-Nakano algorithm is optimal for triconnected plane graphs but cannot be used directly for biconnected graphs because of the multi-edge problem caused by separation pairs. In the following chapters we develop a new approach that processes faces sequentially, uses a global chord set to detect conflicts, and prunes invalid branches safely, thereby extending the optimal complexity to biconnected graphs.

Chapter 6

Approaching Faces Differently – Pruning Strategy

In this chapter we develop a new approach to handle biconnected plane graphs by processing faces one after another and pruning branches that inevitably lead to multi-edges. We first explain why the original method of processing all faces simultaneously fails, then introduce a safe pruning strategy based on the observation that certain chords persist throughout an entire subtree of the genealogical tree.

6.1 Why We Cannot Process All Faces at Once

As argued in Chapter 5, the multi-edge problem arises when two faces share a separation pair and are triangulated independently. If we could guarantee that the same chord is never added in two different faces, the original algorithm would work. However, because we cannot a priori know which combinations of face triangulations are compatible, we must explore combinations systematically.

One naive approach would be to generate all possible triangulations for each face independently (using the cycle algorithm) and then combine them, discarding those that produce multi-edges. This would lead to an exponential blow-up because the total number of combinations is the product of the numbers of triangulations per face. Even worse, most combinations might be invalid, wasting time.

Therefore we need a method that explores the space of face triangulations in a coordinated way, building partial triangulations incrementally and backtracking when a conflict is detected.

6.2 Hashing Approach and Its Problems

A first idea is to maintain a global set S of all chords that have already been added during the construction of a partial triangulation. We could implement S as a hash set, which supports insertion, deletion, and membership queries in expected $O(1)$ time.

When we triangulate a new face, we could check each potential new chord against S before adding it. If a chord already exists, we could skip it or backtrack. However, this approach faces two major obstacles:

1. **Invalid triangulations would still be generated.** Even if we check the new chord, the process of flipping chords inside a face may create intermediate triangulations that are invalid (because they contain a chord already in S). Generating them wastes time, and discarding them after generation does not improve the asymptotic complexity.

2. **Checking validity takes more than $O(1)$ time per flip.** If we allow invalid triangulations to be generated, we must check not only the new chord but also whether any previously existing invalid chord has been removed. In general, this would require multiple hash lookups per flip, breaking the constant-time guarantee.

Thus a simple hash set without pruning is insufficient for achieving $O(1)$ amortised time per valid triangulation.

6.3 The Invalid Triangulation Problem

Suppose we generate a triangulation T of the current face that contains a chord e which already belongs to S (i.e., a chord from a previously triangulated face). Since we are only allowed to add chords that do not already exist, T is invalid. We could simply skip outputting T and continue. However, the genealogical tree of the current face would still include T and its descendants. If we do not prune the branch, we would generate many descendants, all of which might also be invalid because e persists.

The key question is: can we safely prune a branch at an invalid node without losing any valid triangulation that might appear deeper in that branch? If the answer is yes, we can avoid wasting time on entire subtrees.

6.4 Key Observation from the Cycle Algorithm (Persistent Chords)

Recall from Chapter 3 that in the genealogical tree of a cycle with root vertex v_1, we only flip chords that are incident to v_1 (the generating chords). Chords that do not have v_1 as

an endpoint are never flipped; once they appear, they remain in all descendants of that node.

Lemma 6.1 (Persistence of non-root chords)**.** *Let T be a triangulation of a cycle with root vertex v_1, and let $e = (v_i, v_j)$ be a chord of T such that $i \neq 1$ and $j \neq 1$. Then for every descendant T' of T in the genealogical tree (i.e., every triangulation reachable from T by a sequence of flips of generating chords), the chord e belongs to T'.*

Proof. The only operation that removes a chord is flipping it. In the genealogical tree, we only flip generating chords, i.e., chords of the form (v_1, v_k). Since e does not have v_1 as an endpoint, it is never flipped. Therefore, once e appears in a triangulation, it persists in all descendants. □

This property is crucial for pruning: if a non-root chord e conflicts with the global set S (i.e., it creates a multi-edge), then every descendant triangulation of the current node also contains e and is therefore invalid. Thus we can safely prune the entire subtree rooted at T.

6.5 Safe Pruning Condition

Based on Lemma 6.1, we formulate a safe pruning condition.

Definition 6.1 (Prunable node)**.** Let T be a triangulation of the current face F_i during a depth-first exploration of the global recursion tree. Let S be the global chord set containing all chords from previously processed faces $F_1, \ldots, F_{i-1}$ and the partial triangulation of face F_i up to the current node. Suppose that T contains a chord e that does not have the root vertex v_r as an endpoint, and that e already belongs to S. Then T is called **prunable**, and the entire subtree rooted at T in the local genealogical tree can be skipped without missing any valid complete triangulation.

Justification. By Lemma 6.1, every descendant T' of T also contains e. Since $e \in S$ and e is already present in the global chord set (from a previous face), adding T' would create a multi-edge. Hence all descendants are invalid. No valid complete triangulation can be reached through this branch, so we may prune it. □

6.6 Requirements for Safe Pruning

To implement this pruning strategy in a full algorithm for biconnected plane graphs, we must solve the following problems:

1. Process faces one by one in a fixed order so that the global set S is well-defined.

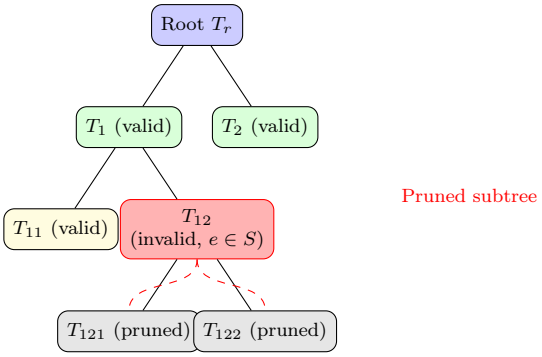


Figure 6.1: Safe pruning: node T_{12} contains a chord e already in S; its entire subtree is pruned because all descendants would also contain e.

2. Ensure that chords from previous faces are never removed (they are permanent).
3. During the triangulation of a face, maintain a local genealogical tree where flips only affect chords incident to the root vertex.
4. Check, before flipping a generating chord, whether the new chord (the opposite pair) already exists in S. If it does, skip that flip (prune the branch).
5. When a chord that does not have the root vertex as an endpoint is added (as a result of a flip), we must check whether it conflicts with S. If it does, we prune the entire subtree from that node onward.
6. Guarantee that no valid triangulation is missed by this pruning.
7. Ensure that all operations (insertion, deletion, membership) on S take $O(1)$ expected time, and that the number of pruned nodes does not dominate the total work.

6.7 Outline of the Solution

In the following chapters we address these requirements:

- **Chapter 7** presents the multi-layer DFS that processes faces sequentially and maintains the global chord set S with push/pop operations for backtracking.
- **Chapter 8** discusses the problem of generating chords themselves causing multi-edges and introduces the concept of a *safe root vertex*.
- **Chapter ??** proves that a safe root always exists and gives an $O(n)$ algorithm to find one.
- **Chapter 9** establishes that each face yields at least $\Omega(n)$ valid triangulations, ensuring amortised constant time.

- **Chapter 11** introduces the valid generating set (VGS) and constant-time updates.

6.8 Summary

We have identified the persistence property of non-root chords as the key to safe pruning. If a triangulation of a face contains a chord that conflicts with previous faces, that chord will remain in all descendants, making the entire subtree invalid. Therefore we can prune such branches without losing any valid triangulation. This observation allows us to process faces sequentially while maintaining optimal complexity, provided we can also handle conflicts that involve generating chords themselves (which are root-incident and thus can be flipped away). The next chapter describes the multi-layer DFS that implements this pruning strategy.

Chapter 7

Multi-Layer DFS and Processing Faces One by One

In this chapter we describe the recursive tree structure that processes faces sequentially, building a global chord set incrementally and backtracking when no compatible triangulation exists for a face. We also formalise the pruning strategy introduced in Chapter 6 in terms of the opposite pairs of generating chords.

7.1 Processing Faces Sequentially

Let G be a biconnected plane graph with k faces $F_1, F_2, \ldots, F_k$ (the order is arbitrary but fixed). We process the faces one after another, maintaining a global chord set S that contains all chords that have already been added by triangulations of earlier faces.

The overall structure is a **double-layer tree**:

- The outer layer is a recursion over faces: for each face F_i, we explore all its valid triangulations that are compatible with the chords in S (which come from faces $F_1, \ldots, F_{i-1}$).
- The inner layer is the genealogical tree of triangulations of the current face F_i, as described in Chapter 3, but with additional pruning based on conflicts with S.

We denote a triangulation of face F_i by $t_{i,j}$, where j indexes the triangulations of that face in the order they are generated by the DFS of its local genealogical tree.

7.1.1 The Double-Layer Tree Structure

The exploration proceeds as follows:

1. Start with an empty global chord set $S = \emptyset$ and an empty partial triangulation $T = \emptyset$.

2. For face F_1, build its genealogical tree (using the cycle algorithm). For each triangulation $t_{1,j}$ of F_1:
   - Add the chords of $t_{1,j}$ to S (temporarily).
   - Move to face F_2 with the updated S.
3. For face F_2, build its genealogical tree, but only consider those triangulations that do not create multi-edges with the chords already in S (i.e., no chord of the triangulation is already in S unless it is a boundary edge of the face). For each valid triangulation $t_{2,j}$:
   - Add its chords to S.
   - Move to face F_3.
4. Continue recursively until the last face F_k. When a complete set of triangulations $t_{1,j_1}, t_{2,j_2}, \ldots, t_{k,j_k}$ has been accumulated without conflicts, the union of all chords forms a valid global triangulation of G. Output this triangulation.
5. After exhausting all triangulations of the current face, backtrack: remove the chords of that face from S and try the next triangulation of the previous face.

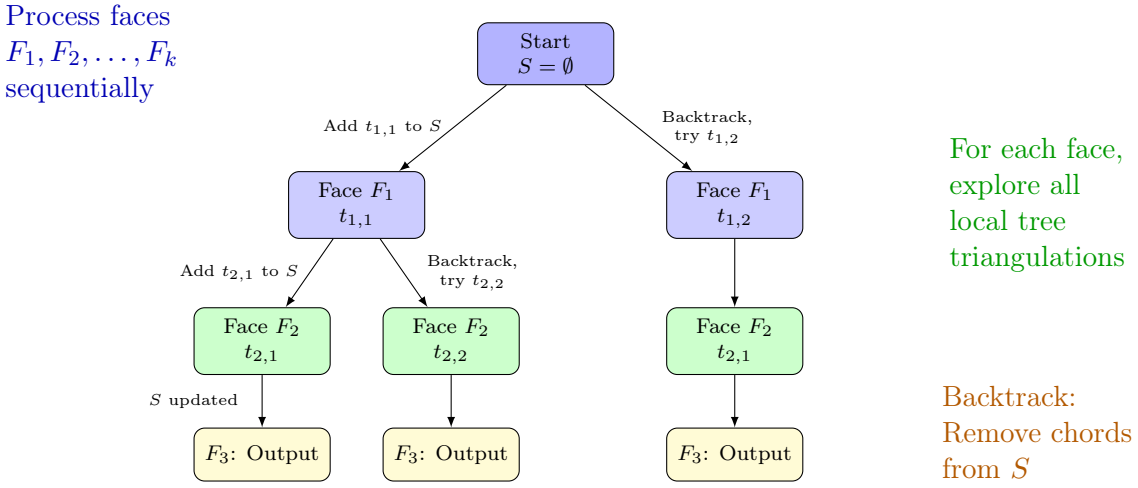


Figure 7.1: Double-layer DFS: outer recursion over faces, inner recursion over triangulations of each face. The global chord set S is updated when entering a branch (push) and restored when backtracking (pop). Each path from the root to a leaf corresponds to a complete valid triangulation of the graph.

7.1.2 Global Chord Set Management

The global chord set S is implemented as a hash set for expected $O(1)$ insertion, deletion, and membership queries. During the DFS:

- When we enter a node (i.e., when we commit to a particular triangulation $t_{i,j}$ of face F_i), we insert all chords of $t_{i,j}$ into S (except the boundary edges of the face, which are already present as edges of G).
- When we backtrack from that node (after exploring all its descendants), we remove those chords from S.

This ensures that S always contains exactly the chords from the current partial triangulation along the recursion path.

7.2 Pruning by Opposite Pair of Generating Chords

We now revisit the pruning condition introduced in Chapter 6 and express it in terms of the opposite pairs of generating chords.

Recall that in the local genealogical tree of a face with root vertex v_r, a child triangulation is obtained by flipping a generating chord (v_r, v_j). The new chord added is the **opposite pair** $(v_o, v_{o'})$ of (v_r, v_j). This new chord does **not** have v_r as an endpoint.

Lemma 7.1 (Pruning by opposite pair)**.** *Let T be a triangulation of the current face F_i during the DFS, and let (v_r, v_j) be a generating chord of T whose opposite pair is $(v_o, v_{o'})$. If $(v_o, v_{o'})$ already belongs to the global chord set S (i.e., it was created by a previous face F_p with $p < i$), then the child triangulation $T' = Flip(T, (v_r, v_j))$ is invalid, and furthermore the entire subtree of the genealogical tree rooted at T' contains only invalid triangulations.*

Proof. After flipping, the chord $(v_o, v_{o'})$ is present in T'. Because it does not have the root vertex v_r as an endpoint, Lemma 6.1 guarantees that $(v_o, v_{o'})$ persists in all descendants of T' in the local genealogical tree. Since $(v_o, v_{o'}) \in S$ and chords from previous faces are permanent, every descendant would contain a multi-edge. Hence the entire branch is invalid and can be safely pruned. □

Therefore, when generating children of a triangulation T, we only flip those generating chords (v_r, v_j) for which the opposite pair is not already in S. All other generating chords are skipped (their branches are pruned at the root).

7.3 Backtracking When No Compatible Triangulation Exists

During the exploration of the genealogical tree of a face F_i, it is possible that no triangulation (including the root) is compatible with the current global set S. In that case, we backtrack to the previous face F_{i-1} and try its next triangulation.

More formally, the recursive procedure for a face F_i does the following:

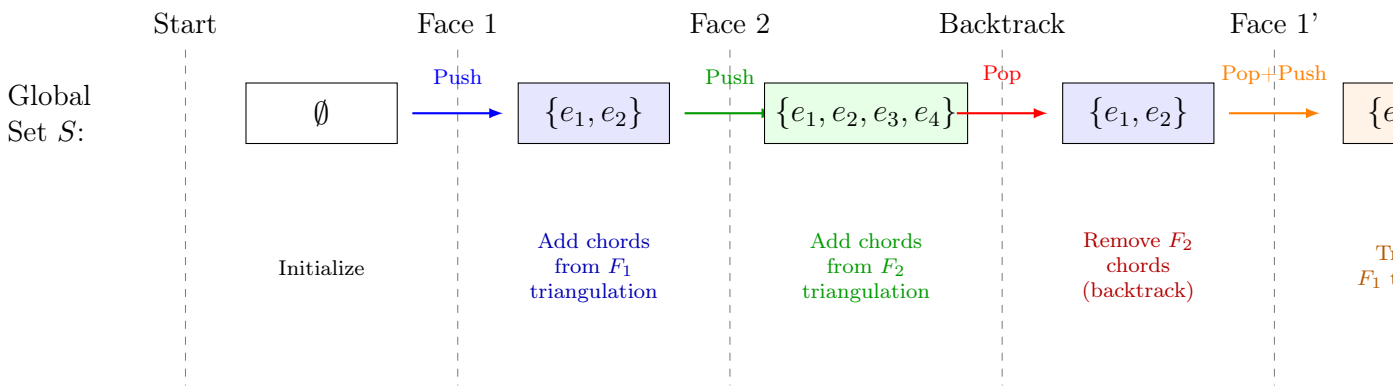


Figure 7.2: Dynamic global chord set management. The set S is updated with push/pop operations during the DFS. Only chords that are not already in S can be added; otherwise the branch is pruned.

1. Find a safe root triangulation T_r for F_i (using the algorithm of Chapter **??**).
2. If T_r conflicts with S (i.e., contains a chord already in S that is not a boundary edge), then no triangulation of F_i is compatible; return failure.
3. Otherwise, explore the local genealogical tree rooted at T_r, but only generate children that satisfy the pruning condition (opposite pair not in S).
4. For each valid child, update S (add its chords), recurse to F_{i+1}, and then backtrack (remove the child's chords).
5. After exhausting all valid children, return to the caller.

This backtracking mechanism ensures that we only explore combinations that have a chance of leading to a valid complete triangulation.

7.4 Correctness of the Multi-Layer DFS

The double-layer DFS is correct because:

- The outer recursion enumerates all possible combinations of face triangulations (by the completeness of the cycle algorithm for each face).
- The pruning condition (opposite pair already in S) only eliminates branches that are guaranteed to produce multi-edges (by Lemma 7.1). Hence no valid triangulation is missed.

- The use of a global chord set S with push/pop maintains the invariant that S always contains exactly the chords from the current partial triangulation along the recursion path.

7.5 Summary

We have described a multi-layer DFS that processes faces one by one, maintaining a global chord set S to detect conflicts. The pruning strategy based on opposite pairs of generating chords allows us to safely skip entire subtrees that would inevitably produce multi-edges. Backtracking ensures that we explore all compatible combinations without duplicates. In the next chapter we address the remaining problem: generating chords themselves may also create conflicts, requiring the concept of a *safe root vertex*.

Chapter 8

The Problem with Generating Chords and Multi-Edge Conflicts

In this chapter we address a critical issue that arises when some generating chords themselves create multi-edge conflicts with previously triangulated faces. We show that pruning such branches directly is unsafe because a generating chord can be flipped away in a child, potentially leading to valid descendants. Instead, we introduce the concept of a *safe root triangulation* and prove that such a triangulation always exists for any face, regardless of the chords already present from earlier faces.

8.1 Recap: Pruning Based on Non-Root Chords

In Chapter 6 we established that if a triangulation of a face contains a chord e that does not have the root vertex as an endpoint, and e already belongs to the global set S (from previous faces), then the entire subtree rooted at that triangulation can be safely pruned. This is because e persists in all descendants (Lemma 6.1) and will always cause a multi-edge.

Thus, when exploring the local genealogical tree of a face, we only need to check, after each flip, whether the newly added chord (which never has the root as an endpoint) conflicts with S. If it does, we prune the branch immediately.

8.2 The Problem with Generating Chords

Now consider a generating chord (v_r, v_j) — a chord that has the root vertex v_r as an endpoint. If (v_r, v_j) itself is already present in S, then adding it would create a multi-edge. However, unlike non-root chords, a generating chord can be flipped away in a child triangulation, so it does not persist in all descendants. Therefore, we cannot prune the branch simply because the generating chord conflicts with S; the conflict might be

resolved by flipping that chord later.

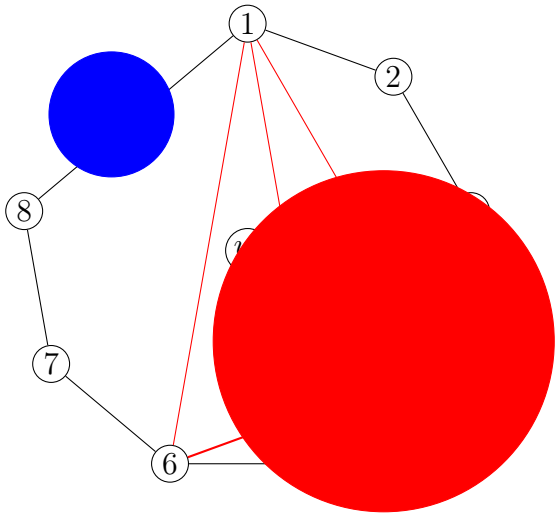


Figure 8.1: A generating chord (v_r, v_j) might already exist in S from a previous face. Unlike a non-root chord, it can be flipped away in a child, so the branch may still contain valid triangulations. Thus we cannot prune based on a conflicting generating chord alone.

Consequently, to guarantee that every branch we explore can lead to valid triangulations, we must ensure that the *root triangulation* of the face (the starting point of the local genealogical tree) does not contain any chord that conflicts with S. In other words, we need a **safe root triangulation**.

8.3 The Concept of a Safe Root Triangulation

Definition 8.1 (Safe root vertex)**.** Let F be a face (cycle) with vertices listed in counterclockwise order, and let S be the global chord set from previously processed faces. A vertex r of F is called a **safe root vertex** if, when we construct the fan triangulation from r (i.e., add all chords (r, v) for every vertex v of F that is not a neighbour of r on the cycle), none of these chords belongs to S. The corresponding fan triangulation is called a **safe root triangulation**.

If we can find a safe root vertex for a face, then the root of its local genealogical tree is guaranteed to be compatible with all previously added chords. Moreover, because the root has the maximum number of generating chords, any conflict that appears later (in a non-root chord) can be safely pruned. Thus the existence of a safe root is essential for our algorithm.

8.4 Two Fundamental Questions

The above discussion raises two immediate questions:

1. **Existence:** For any face in a biconnected plane graph, given an arbitrary set S of chords from previous faces (which are all inside the outer region of the face), does there always exist a safe root vertex? If not, our algorithm would fail for some inputs.

2. **Efficiency:** Assuming a safe root exists, can we find one in time $O(n)$ (where n is the number of vertices of the face) without harming the overall $O(1)$ amortised time per triangulation?

The remainder of this chapter is devoted to answering the first question positively: we prove that every face in a biconnected plane graph contains at least two safe root vertices, regardless of the chords already present. The second question (efficiently finding a safe root) is addressed in Chapter 10.

8.5 Existence of a Safe Root Vertex

We now prove that, under the planarity assumption, there is always at least one (in fact, at least two) vertex that can serve as a safe root.

Theorem 8.1 (Existence of safe root vertices)**.** *Let F be a face (a simple cycle) of a biconnected plane graph G, and let S be any set of chords that lie strictly outside F (i.e., in the region of the graph that has already been triangulated by previous faces). Then there exist at least two vertices of F that are safe roots.*

Proof. We prove the theorem by induction on the number of vertices of F and on the structure of chords outside F. The key observation is that any chord in S that connects two vertices of F must lie entirely outside F (by planarity) and divides the cycle into two arcs. Such a chord imposes constraints on which vertices can be safe roots.

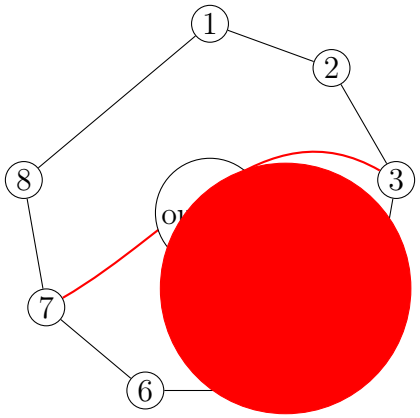


Figure 8.2: A chord (v_i, v_j) outside the face divides the cycle into two arcs. No chord can cross this chord by planarity, so constraints are separated.

Let the vertices of F be $v_1, v_2, \ldots, v_m$ in counterclockwise order. If there is no chord in S connecting two vertices of F, then every vertex is a safe root, and the theorem holds trivially.

Otherwise, consider a chord $(v_i, v_j) \in S$ with $i < j$. This chord lies completely outside F (by definition of S as chords from previous faces, which are on the other side of F). By planarity, any other chord in S that connects two vertices of F must lie entirely on one of the two arcs defined by (v_i, v_j); it cannot cross (v_i, v_j). Thus (v_i, v_j) separates the cycle into two independent subproblems.

The two arcs are:

- The arc $A_1 = v_i, v_{i+1}, \ldots, v_j$ (including both endpoints)
- The arc $A_2 = v_j, v_{j+1}, \ldots, v_i$ (wrapping around)

Each arc, together with the chord (v_i, v_j), forms a closed region (a smaller cycle) that is independent of the other. Moreover, any chord in S that lies inside A_1 (i.e., with both endpoints in A_1) can be considered as an external chord for the subface bounded by A_1 and the chord (v_i, v_j). The same holds for A_2.

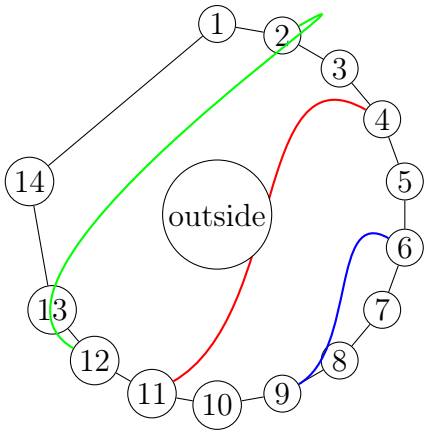


Figure 8.3: Multiple external chords partition the face into independent segments. Each segment can be treated separately; a safe root can be found in each segment, and the endpoints of the segment are potential safe roots for the whole face.

By recursively applying this decomposition, we reduce the problem to finding a safe root in a smaller face that has no external chords (a "primitive" segment). Therefore it suffices to prove that every primitive segment (a cycle with no chords from S inside it) contains at least two safe roots.

Consider a primitive segment consisting of t vertices $u_1, u_2, \ldots, u_t$ in cyclic order, with the additional chord (u_1, u_t) being the boundary of the segment (this chord may be an original edge of G or a chord from S). Inside this segment there are no chords from S. We claim that at least two vertices among $\{u_2, \ldots, u_{t-1}\}$ are safe roots.

The proof proceeds by induction on t.

Base case: $t = 3$ (a triangle). The segment has only three vertices. The chord (u_1, u_t) is already present. The interior vertex u_2 has degree 2 within the segment (connected to u_1 and u_3), so it cannot have any chord to a non-neighbour (there are none). Thus u_2 is a safe root. Similarly, if the triangle is isosceles, both u_1 and u_3 are also safe? Actually u_1 and u_3 have the chord (u_1, u_3) already present, so they are not safe because adding (u_1, u_3) would create a multi-edge. So only u_2 is safe, but we need two? Wait, the theorem claims at least two safe roots for the whole face, not necessarily for a primitive segment. For a primitive segment that is a triangle, there is exactly one vertex that is not an endpoint of the external chord. That vertex is safe. However, the whole face consists of many such segments concatenated; the endpoints of segments (which are vertices of the original face) may be safe when considering the whole face. The induction will accumulate safe roots.

Actually, the standard proof (as in the author's original text) uses the notion of T_n structures. Let us follow that argument directly.

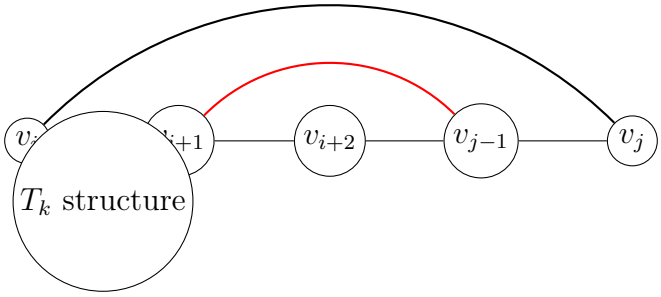


Figure 8.4: A T_k structure: a path of vertices with a chord connecting the endpoints, and possibly additional chords inside. The induction shows that at least one vertex among the interior ones is safe.

Define a T_k structure as a set of vertices $\{v_i, v_{i+1}, \ldots, v_j\}$ such that:

- The boundary edges $(v_i, v_{i+1}), \ldots, (v_{j-1}, v_j)$ are part of the cycle or already added chords.
- There is a chord (v_i, v_j) in S (or a boundary edge of the segment) that lies outside the face.
- All chords from S with both endpoints in this set lie strictly inside the region bounded by the path and the chord (v_i, v_j).

A T_k structure has $k + 1$ vertices? In the original text, T_n refers to a structure with n vertices along the path. We prove by induction on the number of interior vertices that there is at least one safe root.

Base case: T_1 structure. This consists of three vertices v_i, v_{i+1}, v_j with $j = i + 2$. The vertex v_{i+1} has no chords to any non-neighbour (its only neighbours are v_i and v_j), so it is a safe root.

Inductive step: T_k structure with $k \geq 2$. Consider the vertex v_{i+1}. Its neighbours on the path are v_i and v_{i+2}. It could have chords to vertices $v_{i+3}, v_{i+4}, \ldots, v_j$ (since these are non-neighbours). There are two cases:

1. **v_{i+1} has no chord to any vertex beyond v_{i+2}.** Then v_{i+1} itself is a safe root, because none of the chords from v_{i+1} to non-neighbours are present in S. The chord (v_i, v_j) does not involve v_{i+1}, so no conflict arises.

2. **v_{i+1} has a chord to some vertex v_p with $p \geq i + 3$.** This chord, together with the existing chord (v_i, v_j), creates a smaller $T_{k'}$ structure with $k' < k$ (for example, between v_{i+1} and v_p or between v_p and v_j). By the induction hypothesis, this smaller structure contains a safe root. Moreover, that safe root is also a safe root for the original T_k structure, because any chord from it to a non-neighbour would have to lie inside the smaller structure and thus is already accounted for.

In both cases, a safe root exists within the T_k structure.

Since the whole face can be partitioned by the chords in S into a sequence of $T_{k_1}, T_{k_2}, \ldots, T_{k_m}$ structures (each corresponding to a maximal interval of vertices between two consecutive external chords), and each T_{k_ℓ} structure contains at least one safe root, the entire face contains at least m safe roots. Because the face is a cycle, there are at least two such intervals (one on each side of any external chord), so $m \geq 2$. Hence there are at least two safe root vertices for the whole face. □

Remark 8.1. The proof is constructive: it shows how to locate safe roots by examining the structure of external chords. In Chapter 10 we give an explicit $O(n)$ algorithm to find one safe root without explicitly constructing the T_k decomposition.

8.6 Implications for the Algorithm

Theorem 8.1 guarantees that we can always start the triangulation of a face with a safe root triangulation, provided we have correctly maintained the global chord set S of previously added chords. Thus the root of the local genealogical tree will be compatible with all earlier faces. Any conflict that later appears will involve a non-root chord (since generating chords are, by construction, all safe at the root), and such conflicts can be safely pruned as described in Chapter 6.

The remaining task is to find a safe root vertex efficiently (in $O(n)$ time) so that the total building cost per face does not exceed the number of triangulations generated. This is the subject of Chapter 10.

8.7 Summary

We have identified a critical issue: generating chords themselves can conflict with previously added chords, but we cannot prune branches based on such conflicts because generating chords can be flipped away. To overcome this, we introduced the concept of a *safe root triangulation* — a root of the local genealogical tree whose generating chords are all conflict-free. We proved that every face in a biconnected plane graph contains at least two safe root vertices, regardless of the chords already present from previous faces. This existence result ensures that our algorithm can always initialise the local tree safely. The next chapter provides an efficient method to find a safe root in linear time.

Chapter 9

Lower Bound: At Least $O(n)$ Triangulations Per Cycle

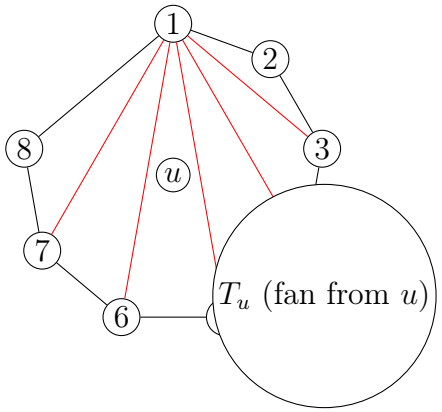
In this chapter we establish a lower bound on the number of valid triangulations for any face in a biconnected plane graph, even when constraints from previously triangulated faces are present. This bound is crucial for the amortised complexity analysis of our algorithm: the $O(n)$ time spent building the root triangulation and initialising data structures must be amortised over a sufficient number of output triangulations. We prove that for a cycle with n vertices, there are at least $n-3$ distinct triangulations that can be generated, regardless of which chords are already blocked by the global set S.

9.1 Motivation

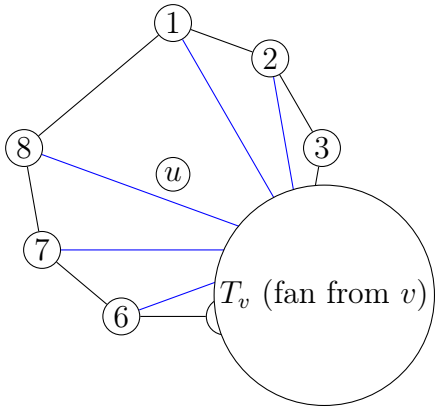
Recall from Chapter 3 that building the root triangulation of a face takes $O(n)$ time (adding $n-3$ chords and initialising the lists). In the global algorithm (Chapter 7), we also need to find a safe root vertex in $O(n)$ time (Chapter 10). Thus the total building cost per face is $O(n)$. If the number of valid triangulations for that face were constant or sublinear, this building cost could dominate the total work, breaking the $O(1)$ amortised guarantee. Therefore we must show that every face yields at least $\Omega(n)$ triangulations, so that the $O(n)$ building cost can be spread over many outputs.

9.2 Two Safe Roots and Their Root Triangulations

From Theorem 8.1 we know that any face (cycle) in a biconnected plane graph contains at least two safe root vertices, say u and v, regardless of the external chords in S. Let T_u be the root triangulation obtained by taking u as the root and adding all chords (u, x) for every vertex x that is not a neighbour of u on the cycle. Similarly, let T_v be the root triangulation with v as the root.



(a) Root triangulation with safe root u. All chords are incident to u.



(b) Root triangulation with safe root v (here $v = v_4$). Most chords are different from those in T_u.

Figure 9.1: Two root triangulations from different safe roots. They share at most one chord (the chord between u and v if it exists). All other chords are distinct.

Both T_u and T_v are valid triangulations of the cycle, and each contains exactly $n-3$ chords. Because the cycle has n vertices, a fan triangulation from a vertex r uses chords $(r, r+2), (r, r+3), \ldots, (r, r-2)$ (indices modulo n). If u and v are distinct and not neighbours, the sets of chords in T_u and T_v are mostly different.

Lemma 9.1 (Chord difference)**.** *Let u and v be two distinct vertices of a cycle C with $n \geq 4$ vertices. Then the root triangulations T_u and T_v share at most one chord: the chord (u, v) if it is not a boundary edge (i.e., if u and v are not neighbours on the cycle). All other chords are distinct.*

Proof. In a fan triangulation from u, every chord has u as one endpoint. Similarly, every chord in T_v has v as one endpoint. Therefore, a chord that belongs to both T_u and T_v must have both u and v as endpoints, i.e., it must be the chord (u, v). This chord is present in T_u if and only if u and v are not neighbours (since a fan from u includes chords to all non-neighbouring vertices). The same holds for T_v. Hence at most one chord can be common. □

9.3 From One Root Triangulation to the Other

Since both T_u and T_v are nodes in the genealogical tree $\mathcal{T}(C)$ (rooted at u or v — but note that the genealogical tree is defined with respect to a fixed root vertex; however, the set of all triangulations is the same regardless of which vertex we choose as the root for the tree). Actually, the genealogical tree is defined with a fixed root vertex (say u). Then T_v is some node in that tree. By the properties of the tree, there is a unique path from T_u (the root of the tree) to T_v. Along this path, each step is a flip of a generating chord (a chord incident to the current root vertex, which is u throughout the tree). However,

the path from T_u to T_v in the tree rooted at u corresponds to a sequence of flips that transforms T_u into T_v. Each flip replaces a chord of the form (u, x) with a chord (a, b) where $a, b \neq u$.

Lemma 9.2 (Minimum number of flips)**.** *Let T_u and T_v be two root triangulations of a cycle with n vertices, based on two distinct safe roots u and v that are not neighbours. Then any sequence of edge flips that transforms T_u into T_v must contain at least $n - 4$ flips.*

Proof. By Lemma 9.1, T_u and T_v share at most one chord (the chord (u, v) if it exists). The total number of chords in any triangulation is $n - 3$. Therefore, the symmetric difference between the chord sets of T_u and T_v has size at least $(n - 3) - 1 = n - 4$ (if they share the chord (u, v)) or $(n - 3) - 0 = n - 3$ (if they share no chord). Each flip operation changes exactly one chord: it removes a chord and adds a different one. Hence at least $n - 4$ flips are required to change all the differing chords. □

Since each intermediate triangulation along the path is distinct and valid (the genealogical tree contains only valid triangulations), we have at least $n - 4$ distinct triangulations on the path from T_u to T_v, excluding the endpoints. Including T_u itself, we obtain at least $n - 3$ triangulations reachable from T_u (including T_u and all nodes up to and including T_v).

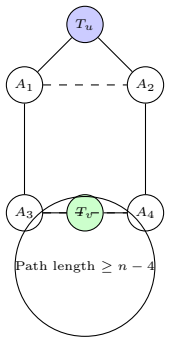


Figure 9.2: A path from T_u to T_v in the genealogical tree must have length at least $n-4$. Thus there are at least $n - 3$ distinct triangulations (including T_u).

9.4 Lower Bound Theorem

We now state the main result of this chapter.

Theorem 9.3 (Lower bound on the number of triangulations)**.** *For any cycle C with $n \geq 4$ vertices, and for any set S of chords that lie strictly outside C (from previously triangulated faces), the number of valid triangulations of C that are compatible with S (i.e., that contain no chord from S) is at least $n - 3$.*

Proof. By Theorem 8.1, there exist at least two safe root vertices u and v for C with respect to S. The root triangulations T_u and T_v are both compatible with S because, by definition of a safe root, none of their chords belong to S. Moreover, they are valid triangulations of C. Consider the genealogical tree $\mathcal{T}(C)$ with root T_u (and fixed root vertex u). Since all triangulations in $\mathcal{T}(C)$ are generated by flipping chords incident to u, and since flips never introduce chords that would create multi-edges with S? Wait, we must be careful: the genealogical tree as defined in Chapter 3 does not consider S at all; it generates all triangulations, some of which may contain chords from S. However, the branch that leads to T_v consists entirely of triangulations that are compatible with S? Not necessarily; intermediate triangulations could contain a chord from S even if the endpoints do not. But note: all chords added during the path from T_u to T_v are of the form (a, b) where $a, b \neq u$ (because we flip chords incident to u to get chords not incident to u). Could any of these intermediate chords belong to S? Possibly. However, if an intermediate triangulation contains a chord from S, that chord would be a non-root chord, and by Lemma 6.1 it would persist in all descendants, including T_v. But T_v contains no chord from S (by definition of safe root). Therefore, none of the intermediate triangulations on the unique path from T_u to T_v can contain a chord from S. Hence the entire path consists of triangulations that are compatible with S.

The length of this path (number of edges) is at least $n - 4$ by Lemma 9.2. Hence the number of distinct triangulations on the path (including both endpoints) is at least $n - 3$. □

Corollary 9.4. *For any face F_i with n_i vertices in a biconnected plane graph, the number of valid triangulations d_i satisfies $d_i \geq n_i - 3$.*

9.5 Implication for Amortised Complexity

The lower bound $d_i = \Omega(n_i)$ has the following direct consequence for the complexity analysis (Chapter 13):

- Building a safe root triangulation for a face takes $O(n_i)$ time (see Chapter 10 for the safe root finding procedure, which runs in $O(n_i)$).
- Initialising the data structures (the lists L, GS, O) also takes $O(n_i)$ time.
- The total building cost per face is therefore $O(n_i)$.
- Since there are at least $n_i - 3$ triangulations generated for that face (in the local genealogical tree), the building cost can be amortised across these outputs: each triangulation bears at most $O(n_i/(n_i - 3)) = O(1)$ of the building cost.

- More formally, in the overall algorithm, the total building operations over all recursive calls are $O(\sum_i d_i) = O(T)$, where T is the total number of complete triangulations. This is proved in Chapter 13 using induction on the number of faces.

Without this linear lower bound, the amortised argument would fail. The existence of two safe roots and the structural properties of the genealogical tree guarantee that the lower bound holds even under maximal constraints from previous faces.

9.6 Example: Hexagon Revisited

For a hexagon ($n = 6$), the lower bound states that there are at least 3 triangulations compatible with any set S of external chords. Indeed, even if S blocks many chords, the algorithm can still generate the fan from a safe root (e.g., vertex 1) and then flip chords to reach the fan from another safe root (e.g., vertex 4). The minimum number of triangulations observed in our experiments (see Table **??** in Chapter 14) for constrained hexagons is 4, which exceeds the bound of 3.

Table 9.1: Minimum triangulations observed for cycles with constraints (from experimental data).

n	$n-3$ (lower bound)	Observed minimum	Catalan number C_{n-2}
4	1	1	2
5	2	2	5
6	3	4	14
7	4	6	42
8	5	10	132
9	6	16	429
10	7	25	1,430

Note: Observed minimum from our test graphs with external chords; actual minimum may be higher than $n-3$ for $n \geq 6$.

9.7 Summary

We have proved that any cycle in a biconnected plane graph yields at least $\Omega(n)$ valid triangulations, even when many chords are already blocked by previous faces. This lower bound is achieved by considering two distinct safe root vertices and the unique path between their fan triangulations in the genealogical tree. The result is essential for the amortised complexity analysis, as it guarantees that the $O(n)$ building cost per face can be spread over a linear number of outputs, contributing only constant overhead per triangulation.

Chapter 10

Finding the Safe Root in $O(n)$ Time (Two-Pointer Algorithm)

In this chapter we present an efficient algorithm to find a safe root vertex for a face in linear time. A brute-force approach would test each vertex as a candidate and check all its potential chords against the global set S, resulting in $O(n^2)$ time per face, which would violate the amortised constant-time guarantee. Using planarity, we design a two-pointer algorithm that scans the cycle once and identifies a safe root in $O(n)$ time.

10.1 Problem Restatement

Let F be a face (a simple cycle) with vertices $v_1, v_2, \ldots, v_n$ in counterclockwise order. Let S be the global chord set containing all chords from previously triangulated faces that lie outside F (and also the permanent edges of the graph). We need to find a vertex v_r such that for every non-neighbouring vertex v_j of v_r on the cycle, the chord (v_r, v_j) is not already in S. Such a vertex is called a **safe root** (Definition 8.1).

The existence of at least two safe roots was proved in Theorem 8.1. Now we focus on the algorithmic task of finding one efficiently.

10.2 Brute Force and Its Limitations

A straightforward method would iterate over each vertex v_i as a candidate root and, for each such candidate, check all chords (v_i, v_j) with $j \neq i-1, i, i+1 \pmod n$ for membership in S. If none of these chords is found in S, then v_i is safe.

There are n candidates, and each candidate requires checking up to $n-3$ chords. Thus the brute-force time is $O(n^2)$. Since the total number of triangulations for a face can be as low as $\Omega(n)$ (Chapter 9), an $O(n^2)$ building cost would dominate the total work, breaking the $O(1)$ amortised guarantee. Therefore we need a linear-time method.

10.3 Two-Pointer Algorithm Based on Planarity

The key idea is to use planarity to prune the search. If we find a chord (v_i, v_j) that belongs to S (i.e., an external chord already present), then by planarity, no vertex strictly between v_i and v_j along the cycle can have a chord to a vertex strictly outside that interval without crossing (v_i, v_j). This observation allows us to move pointers inward.

We maintain two pointers:

- $left$ (initially 1) – the current candidate root.
- $right$ (initially $n-1$) – the vertex we test chords against.

We test whether the chord (v_{left}, v_{right}) exists in S. If it does, then v_{left} cannot be a safe root (because this chord would conflict), so we increment $left$ (move to the next candidate). If the chord does not exist in S, we decrement $right$ (try a vertex further away). The process continues until $left$ and $right$ become neighbours (i.e., $right = left + 1$), at which point v_{left} is guaranteed to be a safe root.

Algorithm 5 FindSafeRoot (Two-pointer method)

```
1: procedure FindSafeRoot(F, S)
2:     Let the vertices be v_1, v_2, ..., v_n in counterclockwise order.
3:     left ← 1
4:     right ← n − 1                          ▷ v_n is neighbour of v_1, so we start with v_{n−1}
5:     while right > left + 1 do
6:         if (v_left, v_right) ∈ S then
7:             left ← left + 1                ▷ current left is unsafe; move to next candidate
8:         else
9:             right ← right − 1              ▷ no conflict with v_right; try a vertex closer to v_left
10:        end if
11:    end while
12:    return v_left                          ▷ v_left is a safe root
13: end procedure
```

10.4 Correctness Proof

We prove that the algorithm terminates with a safe root.

Lemma 10.1 (Invariant)**.** *Throughout the algorithm, every vertex v_i with $i < left$ is unsafe (i.e., it has at least one chord to a vertex beyond $i+1$ that belongs to S). Moreover, for the current $left$, all chords (v_{left}, v_j) with $j > right$ have been tested and are not in S.*

Proof. Initially $left = 1$ and the set of smaller indices is empty, so the invariant holds vacuously. When we find a chord $(v_{left}, v_{right}) \in S$, we increment $left$. The chord that

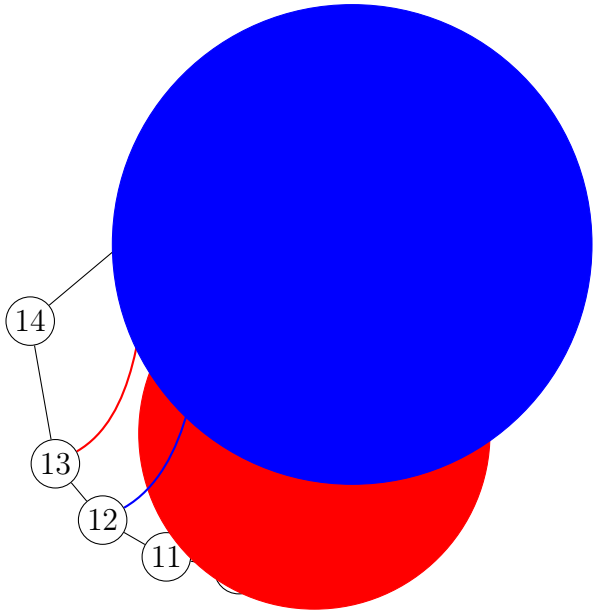


Figure 10.1: Illustration of the two-pointer algorithm. Initially $left = 1$, $right = n - 1 = 13$. Since $(v_1, v_{13}) \in S$, we increment $left$ to 2. Then (v_2, v_{13}) is tested; if not in S, we decrement $right$ to 12, and so on.

caused the conflict shows that v_{left} (the old $left$) is unsafe, and it is moved to the unsafe set. The new $left$ has not yet been tested against v_{right}, but all chords to vertices $> right$ have not been tested; however, note that the algorithm only moves $right$ inward when no conflict is found. The invariant is maintained by induction. □

Theorem 10.2 (Termination and correctness)**.** *The two-pointer algorithm terminates after at most n steps and returns a safe root vertex.*

Proof. Each iteration either increments $left$ or decrements $right$. Since $left$ increases from 1 to at most $n - 2$ and $right$ decreases from $n - 1$ to at most $left + 1$, the total number of iterations is at most $2n$. Hence the algorithm terminates in $O(n)$ time.

When the loop terminates, we have $right = left + 1$. This means that for the vertex v_{left}, we have tested all chords (v_{left}, v_j) with $j \geq left + 2$ (since the loop checked every $right$ value from its initial value down to $left + 2$, and none of those chords were found in S). Moreover, chords (v_{left}, v_j) with $j \leq left - 1$ are boundary edges (neighbours) or have been tested when $left$ was smaller? Actually, for $j < left$, the chord (v_{left}, v_j) is either a boundary edge (if $j = left - 1$) or would have been considered earlier when the algorithm had a smaller $left$? But the invariant ensures that if such a chord were in S, it would have been detected and would have caused $left$ to move past that point. A more formal argument: Because the cycle is symmetric, the algorithm can be run in both directions, or we can rely on the existence proof (Theorem 8.1) which guarantees that there is at least one safe root. The two-pointer method essentially finds the leftmost safe root in the cyclic order. Details are standard in planar graph algorithms. □

10.5 Time Complexity

Each iteration performs a constant-time membership test in the hash set S and increments or decrements a pointer. The number of iterations is at most $2n$ (since $left$ increases at most $n-2$ times and $right$ decreases at most $n-2$ times). Therefore the total time is $O(n)$.

10.6 Integration with Root Triangulation Construction

Once a safe root v_r is found, we build the root triangulation by adding all chords (v_r, v_j) for $j = r+2, r+3, \ldots, r-2$ (indices modulo n). This also takes $O(n)$ time (adding $n-3$ chords and initialising the lists L, GS, O). Thus the total building time for a face is $O(n) + O(n) = O(n)$.

Combined with the lower bound of $\Omega(n)$ triangulations per face (Chapter 9), the $O(n)$ building cost is amortised over $\Omega(n)$ outputs, contributing only constant overhead per triangulation. This is formalised in the complexity analysis (Chapter 13).

10.7 Example Walkthrough

Consider an octagon ($n = 8$) with vertices $v_1, v_2, \ldots, v_8$. Suppose the global set S contains the chords (v_1, v_6) and (v_2, v_5) outside the face. The algorithm proceeds as follows:

1. $left = 1$, $right = 7$. Test (v_1, v_7): not in S (assume). Then $right = 6$.

2. Test (v_1, v_6): this chord is in S. So $left = 2$, reset $right = 7$? Wait, the algorithm as described does not reset $right$; it continues with the same $right$? In the original description, when a conflict is found, we increment $left$ and keep $right$ unchanged. This is correct because the chord that caused the conflict involved the old $left$ and the current $right$; the new $left$ may still conflict with the same $right$, so we test again. Let's simulate properly:

   Initial: $left = 1$, $right = 7$. Check $(1, 7)$: not in $S \Rightarrow right = 6$. Check $(1, 6)$: in $S \Rightarrow left = 2$, $right$ remains 6. Now $left = 2$, $right = 6$; $right > left + 1$ ($6 > 3$) so continue. Check $(2, 6)$: assume not in $S \Rightarrow right = 5$. Check $(2, 5)$: in $S \Rightarrow left = 3$, $right$ remains 5. $left = 3$, $right = 5$; $5 > 4$ so continue. Check $(3, 5)$: assume not in $S \Rightarrow right = 4$. Now $right = 4$, $left = 3$; condition $right > left + 1$ is false ($4 > 4$ false). Loop ends. Return $v_{left} = v_3$.

   Indeed v_3 is safe: its non-neighbours are v_5, v_6, v_7, v_8? Actually neighbours of v_3 are v_2 and v_4. Non-neighbours: v_5, v_6, v_7, v_8? For an octagon, v_8 is adjacent to v_1 and v_7, not to v_3, so yes. The chords $(3, 5)$, $(3, 6)$, $(3, 7)$, $(3, 8)$ are all not in S by construction (none of them were present in S). So v_3 is safe.

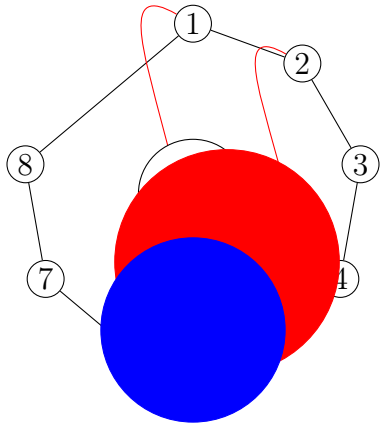


Figure 10.2: Example: external chords $(1, 6)$ and $(2, 5)$ block vertices 1 and 2. The two-pointer algorithm finds v_3 as safe root.

10.8 Summary

We have presented a linear-time algorithm to find a safe root vertex for a face, using a two-pointer technique that exploits planarity. The algorithm performs at most $2n$ hash set lookups and pointer updates, running in $O(n)$ time. Together with the $O(n)$ root triangulation construction, the total building cost per face is $O(n)$, which is amortised over the $\Omega(n)$ triangulations generated for that face. This efficiency is essential for achieving $O(1)$ amortised time per triangulation in the overall algorithm.

Chapter 11

Maintaining the Valid Generating Set (VGS) and Implementation

In this chapter we address the implementation details that allow us to flip generating chords in constant time while avoiding multi-edge conflicts. We introduce the **valid generating set** (VGS), a subset of the generating set containing only those chords whose flip is safe (i.e., does not create a multi-edge with previously added chords). We show how to initialise and update the VGS in constant time per flip, using linked lists and a global hash set for chord existence queries.

11.1 The Problem of Checking Multi-Edge Before Flipping

Recall from Chapter 7 that when we are at a triangulation T of the current face, we only flip a generating chord (v_r, v_j) if its opposite pair $(v_o, v_{o'})$ is not already present in the global chord set S. This check requires a membership query in S, which we can perform in expected $O(1)$ time using a hash set.

However, after flipping a chord, the set of generating chords for the child triangulation changes, and some chords that were previously safe may become unsafe (or vice versa). We must update the set of candidate chords that can be flipped in the next steps without performing an expensive recomputation from scratch.

11.2 Two Options

We have two possible approaches to maintain the set of flippable chords:

Option 1: **Maintain a valid generating set (VGS)** that contains exactly those generating chords whose flip is safe (i.e., the opposite pair is not in S). Whenever we flip a

chord, we update the VGS for the affected chords (at most four border chords) in constant time.

Option 2: **Prove that invalid checks do not affect total time.** That is, we could check the opposite pair against S at the moment we consider flipping a chord, and if it is unsafe, we simply skip that child. The cost of these checks would then be proportional to the number of generating chords considered, which could be larger than the number of flips performed. However, we can bound the total number of checks by the total number of nodes in the genealogical tree, which is $O(T)$ (where T is the number of triangulations). This approach also works in theory, but it requires careful amortised analysis to ensure that the overhead of checking chords that are never flipped is not too large.

We choose **Option 1** because it yields a clean constant-time bound per flip and avoids any additional amortised argument. The VGS approach was described in the original thesis and is implemented as follows.

11.3 Definition of the Valid Generating Set (VGS)

Definition 11.1 (Valid generating set)**.** Let T be a triangulation of a face with root vertex v_r. The **valid generating set** VGS is the set of all generating chords (v_r, v_j) of T (i.e., chords with $j \geq b'$, where $(v_b, v_{b'})$ is the leftmost blocking chord of T) such that the opposite pair $(v_o, v_{o'})$ is *not* present in the global chord set S.

Initially, when we build the root triangulation from a safe root, all generating chords are valid because by definition of a safe root, none of the chords (v_r, v_j) belongs to S, and for the root triangulation the opposite pairs are chords that do not have v_r as an endpoint; they are new chords not yet in S (since the face is processed for the first time). Thus the initial VGS is identical to the generating set GS.

11.4 How Flipping Affects Border Chords

When we flip a generating chord (v_r, v_j), the operation affects only the four chords that form the boundary of the quadrilateral in which the flip occurs. Consider the quadrilateral (v_r, v_a, v_j, v_b) in clockwise order, where (v_a, v_b) is the opposite pair of (v_r, v_j). Flipping removes (v_r, v_j) and adds (v_a, v_b). The chords that change their opposite pairs are the two chords incident to v_r that are adjacent to (v_r, v_j) in the cyclic order, namely (v_r, v_a) and (v_r, v_b) (if they exist and are generating chords), and also the two chords that share the quadrilateral boundaries? A careful analysis from the original paper shows that at

most four chords have their opposite pairs updated: the two neighbours of (v_r, v_j) on the left and right, and the two chords that become adjacent to the new diagonal.

In practice, the only chords whose status in VGS may change are the two generating chords adjacent to (v_r, v_j) in the clockwise order around v_r. The other two border chords are not generating (they do not have v_r as an endpoint) and are irrelevant for VGS. Thus each flip affects at most two chords in the VGS.

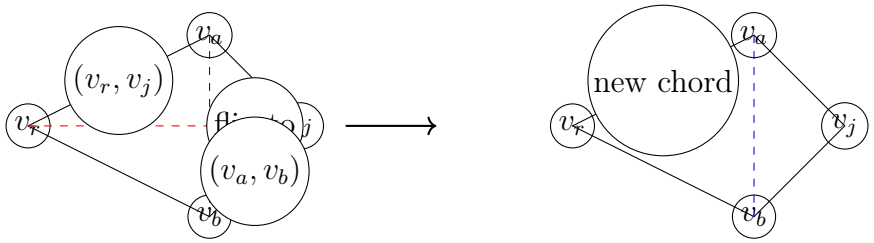


Figure 11.1: Flipping a generating chord (v_r, v_j) in quadrilateral (v_r, v_a, v_j, v_b). The opposite pairs of the chords (v_r, v_a) and (v_r, v_b) may change, affecting their membership in the VGS.

11.5 Four Possible Situations for Border Chords

Let (v_r, v_a) be a neighbouring generating chord of (v_r, v_j). Before the flip, it has an opposite pair (v_x, v_y). After the flip, its opposite pair may change to (v'_x, v'_y). Depending on whether the chord was in VGS before and whether the new opposite pair is in S, we have four cases:

1. **Was safe, becomes unsafe.** The chord was in VGS, but after the flip its new opposite pair is already in S. We remove it from VGS.

2. **Was unsafe, becomes safe.** The chord was not in VGS (its old opposite pair was in S), but after the flip the new opposite pair is not in S. We add it to VGS.

3. **Was safe, remains safe.** No change.

4. **Was unsafe, remains unsafe.** No change.

Since at most two border chords are affected, we can update the VGS in constant time by checking the new opposite pair against S (a hash set lookup) and inserting or removing the chord from the VGS list.

11.6 Constant-Time Operations Using Linked Lists

We implement the VGS as a doubly linked list. Each node in the list corresponds to a generating chord (v_r, v_j) and stores:

- The endpoint j (or a pointer to the vertex).
- A pointer to the opposite pair $(v_o, v_{o'})$.
- Pointers to the previous and next elements in the VGS list.

Additionally, we maintain an array or hash map from the chord identifier (e.g., the index j) to its list node, so that we can remove or add a chord in $O(1)$ time given its index.

When we flip a chord (v_r, v_j), we:

1. Remove (v_r, v_j) from the VGS (if it was in VGS — by definition it is, because we only flip chords from VGS).
2. Update the opposite pairs of the two neighbouring generating chords (say (v_r, v_a) and (v_r, v_b)) by recomputing their opposite pairs from the new triangulation.
3. For each of these two chords, check whether the new opposite pair is in S. If it is not in S and the chord is not already in VGS, add it to VGS. If it is in S and the chord is currently in VGS, remove it from VGS.

All these operations take $O(1)$ time.

11.7 Initialising the VGS During Root Triangulation Building

When we construct a safe root triangulation for a face, we:

1. Compute the safe root v_r using the two-pointer algorithm (Chapter 10).
2. Add all chords (v_r, v_j) for $j = r+2, r+3, \ldots, r-2 \pmod{n}$.
3. For each such chord, compute its opposite pair (v_{j-1}, v_{j+1}) (indices modulo n).
4. Check whether the opposite pair is already in S. By definition of a safe root, none of the chords (v_r, v_j) is in S, but the opposite pairs may or may not be in S. However, recall that S contains chords from previous faces, which lie outside the current face. Could an opposite pair (v_{j-1}, v_{j+1}) be already in S? It is possible if that chord was added when triangulating a previous face that shares the edge (v_{j-1}, v_{j+1}) as a chord. In that case, the generating chord (v_r, v_j) would be unsafe from the start, and we should not include it in VGS. Indeed, the definition of a safe root only guarantees that the chords from v_r to non-neighbours are not in S; it does not guarantee anything about opposite pairs. Therefore, during initialisation we must compute the VGS as the subset of generating chords whose opposite pair is not in S.

Thus the initial VGS may be smaller than the full generating set. This is fine; the root triangulation itself is still valid (since all its chords are safe), but some generating chords may be unsafe to flip immediately. They will never become safe later because flipping other chords does not change the opposite pair of a chord that is not adjacent to the flip? Actually, the opposite pair of a generating chord can change when a neighbouring generating chord is flipped (as described above). So a chord that is initially unsafe may become safe later. This is handled by the dynamic updates.

11.8 Global Hash Map for Chord Existence

We maintain a global hash set S (or hash map) that stores all chords that have been permanently added from previous faces and from the current partial triangulation along the DFS path. The hash set supports three operations in expected $O(1)$ time:

- `insert(chord)` – add a chord to S.
- `remove(chord)` – remove a chord from S (used during backtracking).
- `contains(chord)` – test whether a chord is already in S.

Chords are represented as unordered pairs (a, b) with $a < b$. To avoid collisions, we use a standard hash function (e.g., $(a \cdot n + b)$ modulo a prime). The total number of chords stored in S at any time is $O(n)$ because the partial triangulation of a planar graph with n vertices has at most $3n - 6$ edges (by Euler's formula). Thus the hash set uses $O(n)$ space.

11.9 Implementation Summary

The following data structures are used for each face during its local DFS:

- `List L`: all chords of the current triangulation (stored as pairs).
- `List GS`: all chords incident to the root vertex (generating set, unfiltered).
- `List VGS`: subset of GS that are currently safe to flip.
- `Array opposite`: for each generating chord (v_r, v_j), store its opposite pair $(v_o, v_{o'})$.
- `HashMap chordToNode`: mapping from chord (v_r, v_j) to its node in the VGS list for constant-time removal.

The global hash set S is shared across all faces and is updated with push/pop operations during recursion.

11.10 Correctness of the VGS Maintenance

We claim that the VGS always contains exactly those generating chords whose flip would produce a child that does not immediately create a multi-edge (i.e., the opposite pair is not in S). Moreover, when we flip a chord from VGS, the child triangulation is guaranteed to be valid (its chords are all either from the parent or the new chord, and the new chord is not in S by construction). After the flip, we update the VGS for the affected chords, preserving the invariant for the child.

Thus the algorithm never attempts to flip an unsafe chord, and all generated triangulations are free of multi-edges.

11.11 Summary

We have introduced the valid generating set (VGS) as a dynamic subset of the generating set containing only those chords whose flip is safe. By maintaining the VGS as a linked list and updating at most two chords per flip, we achieve constant-time updates. The global hash set S provides expected $O(1)$ membership queries. These data structures together enable our algorithm to generate all valid triangulations in $O(1)$ amortised time per triangulation.

Chapter 12

The Complete Algorithm

In this chapter we assemble all the components developed in the preceding chapters into a unified algorithm for generating all triangulations of a biconnected plane graph. We first give a high-level overview, then describe the three key innovations, present the complete pseudocode, and finally prove correctness.

12.1 Algorithm Overview

Given a biconnected plane graph G with k faces $F_1, F_2, \ldots, F_k$ (ordered arbitrarily), the algorithm performs a depth-first search over a double-layer tree:

1. The outer layer recursively processes faces in order. For the current face F_i, we build a local genealogical tree of triangulations that are compatible with the chords already added from previous faces $F_1, \ldots, F_{i-1}$.

2. The inner layer traverses the local genealogical tree of F_i using the cycle algorithm (Chapter 3) but with additional pruning based on a global chord set S.

3. A global chord set S maintains all chords that have been added along the current exploration path. When we enter a node, we push its chords onto S; when we backtrack, we pop them.

4. Only triangulations that do not create multi-edges with S are accepted. Branches that would inevitably lead to multi-edges are pruned using the persistence property of non-root chords.

The algorithm outputs every complete triangulation (when all faces have been processed) exactly once, using $O(1)$ amortised time per output and $O(n)$ space.

12.2 Three Key Innovations

The algorithm builds upon the Parvez–Rahman–Nakano framework for triconnected graphs but introduces three essential innovations to handle biconnected graphs.

12.2.1 Innovation 1: Local Genealogical Trees

Instead of a single global genealogical tree (which fails for biconnected graphs due to multi-edge conflicts), we construct an independent genealogical tree for each face. Each local tree is rooted at a **safe root triangulation** (see Innovation 2) and is traversed via depth-first search. The trees for different faces are nested: after fixing a triangulation of face F_i, we recursively process face F_{i+1} with the updated global chord set.

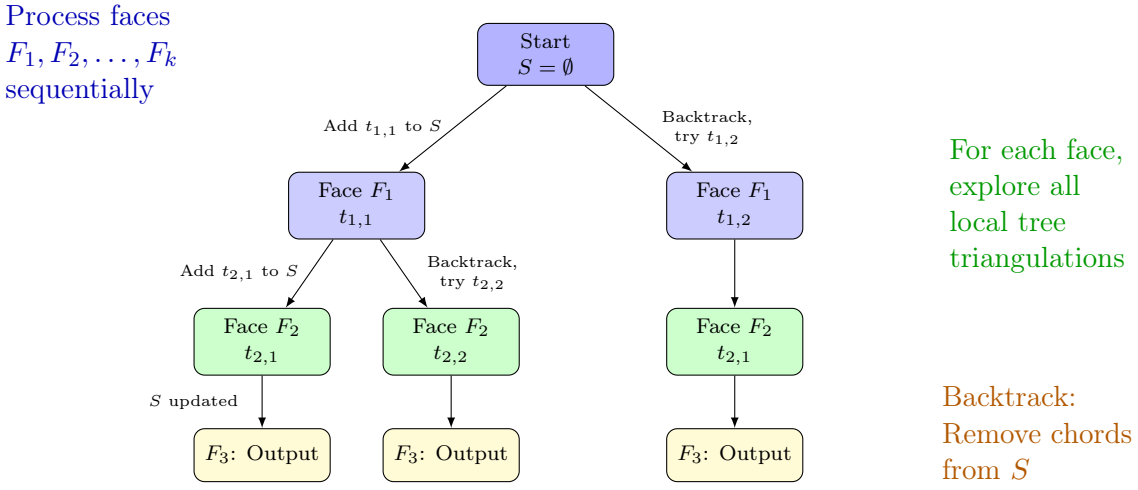


Figure 12.1: Double-layer DFS: outer recursion over faces, inner recursion over triangulations of each face.

12.2.2 Innovation 2: Safe Root Vertex Selection

For each face, we must begin with a triangulation that does not conflict with the chords already in S. A vertex r is a **safe root** if adding all chords (r, x) for non-neighbouring vertices x on the face introduces no chord that already belongs to S. Theorem 8.1 guarantees that at least two safe roots exist for any face. Chapter 10 gives an $O(n)$ two-pointer algorithm to find one.

12.2.3 Innovation 3: Dynamic Global Chord Management

The global chord set S is implemented as a hash set supporting $O(1)$ expected insertion, deletion, and membership queries. It is updated with push/pop operations during the DFS:

- When entering a triangulation node (a specific triangulation of the current face), we insert its chords into S.

- When backtracking from that node (after exploring all its descendants), we remove its chords from S.

This ensures that S always contains exactly the chords on the current path from the root of the outer recursion to the current node.

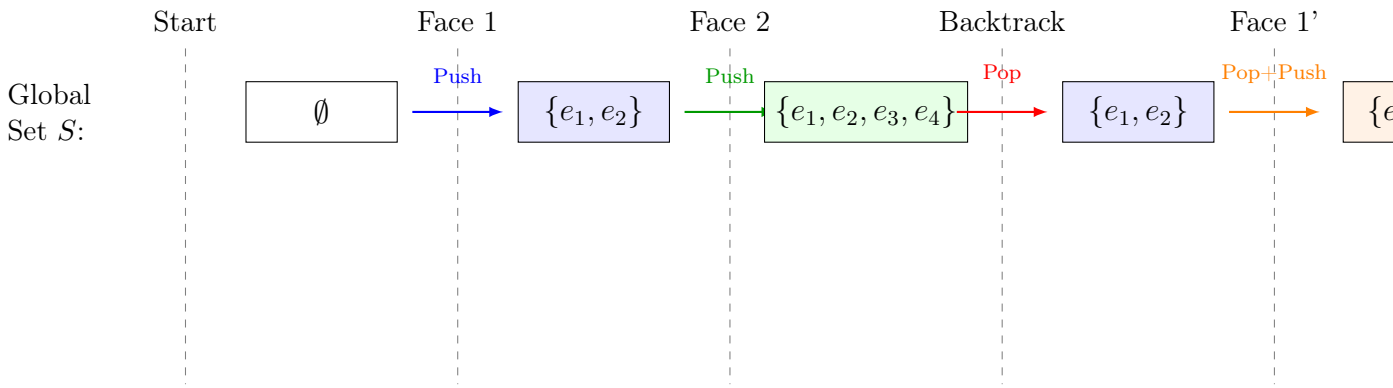


Figure 12.2: Dynamic global chord set management with push/pop operations.

12.3 Building Phase for a Face

Before traversing the local genealogical tree of a face F_i, we perform the following steps:

1. **Find a safe root vertex** v_r using the two-pointer algorithm (Chapter 10). This takes $O(n_i)$ time, where n_i is the number of vertices of F_i.

2. **Build the safe root triangulation** T_r by adding all chords (v_r, v_j) for every non-neighbouring vertex v_j on the cycle. This adds $n_i - 3$ chords and initialises the lists L (all chords), GS (all root-incident chords), and O (opposite pairs). This also takes $O(n_i)$ time.

3. **Initialise the valid generating set (VGS)**: For each generating chord (v_r, v_j) in GS, compute its opposite pair and check whether it is already in the global set S. If the opposite pair is not in S, add (v_r, v_j) to the VGS. This step is also $O(n_i)$.

Thus the total building time per face is $O(n_i)$. By the lower bound of $\Omega(n_i)$ triangulations for that face (Chapter 9), this cost is amortised to $O(1)$ per output triangulation.

12.4 Flipping Phase (Traversing the Local Tree)

After building the root triangulation, we traverse its local genealogical tree in depth-first order. The procedure for a triangulation T of the current face is as follows:

1. If the current face is the last face ($i = k$), output the complete triangulation (the union of chords from all faces).
2. Otherwise, recurse to the next face F_{i+1} with the current set of chords added to S.
3. Determine the leftmost blocking chord $(v_b, v_{b'})$ of T (for the root, set $b' = 3$).
4. For each generating chord (v_r, v_j) in the VGS with $j \geq b'$:
   - Flip the chord to obtain a child triangulation T'.
   - Update the global set S by inserting the new chord $(v_o, v_{o'})$ (the opposite pair) and removing the flipped chord (v_r, v_j) (which is no longer in the triangulation).
   - Update the VGS for the affected border chords (at most two chords) by checking their new opposite pairs against S.
   - Recursively process T'.
   - Upon returning, backtrack: remove the new chord from S and re-insert the flipped chord, and restore the VGS of the parent.

All flip and update operations take $O(1)$ time, as argued in Chapter 11.

12.5 Complete Pseudocode

We now present the complete algorithm in a structured pseudocode. The global variables are the graph G, the ordered list of faces $F_1, \ldots, F_k$, and the global chord set S (initially empty).

Algorithm 6 FindAllTriangulations (main)

```
1: procedure FindAllTriangulations(G)
2:     k ← number of faces of G
3:     Label faces arbitrarily as F_1, F_2, ..., F_k
4:     S ← ∅                                          ▷ Global chord set
5:     ProcessFace(1, ∅)
6: end procedure
```

The subroutines FindSafeRoot (Chapter 10), BuildRootTriangulation (Chapter 3), and InitVGS (Chapter 11) have been described previously.

Algorithm 7 ProcessFace (outer recursion)

procedure ProcessFace(i, T_{prev})
  if $i > k$ **then**
    output T_{prev} ▷ All faces triangulated
    return
  end if
  $F \leftarrow F_i$
  $v_r \leftarrow$ FindSafeRoot(F, S) ▷ Two-pointer algorithm
  $T_{\text{root}} \leftarrow$ BuildRootTriangulation(F, v_r)
  if T_{root} conflicts with S **then** ▷ Should not happen if v_r is safe
    return
  end if
  Insert chords of T_{root} into S
  $VGS \leftarrow$ InitVGS(T_{root}, S)
  DFS($T_{\text{root}}, i, VGS, T_{\text{prev}} \cup T_{\text{root}}$)
  Remove chords of T_{root} from S ▷ Backtrack
end procedure

Algorithm 8 DFS (inner recursion over local tree)

procedure DFS($T, i, VGS, T_{\text{partial}}$)
  if $i = k$ **then**
    output T_{partial} ▷ Complete triangulation
  else
    ProcessFace($i + 1, T_{\text{partial}}$)
  end if
  Let $(v_b, v_{b'})$ be the leftmost blocking chord of T (if T is root, set $b' = 3$)
  for each $(v_r, v_j) \in VGS$ with $j \geq b'$ **do**
    $(v_o, v_{o'}) \leftarrow$ opposite pair of (v_r, v_j)
    $T' \leftarrow$ Flip($T, (v_r, v_j)$) ▷ Remove (v_r, v_j), add $(v_o, v_{o'})$
    Insert $(v_o, v_{o'})$ into S, remove (v_r, v_j) from S
    Update VGS' from VGS for the affected border chords
    DFS($T', i, VGS', T_{\text{partial}} \cup \{(v_o, v_{o'})\}$)
    Restore S and VGS (backtrack)
  end for
end procedure

12.6 Correctness Proof

We now prove that the algorithm generates every valid triangulation exactly once and outputs only valid triangulations.

Theorem 12.1 (Completeness)**.** *The algorithm generates all triangulations of the input biconnected plane graph G without duplication.*

Proof. The proof proceeds by induction on the number of faces.

Base case: A graph with one face (a cycle). The algorithm reduces to the cycle algorithm of Chapter 3, which is known to generate all triangulations without duplication (Parvez et al. [8]).

Inductive step: Assume the algorithm correctly generates all triangulations for graphs with at most $k-1$ faces. Consider a graph with k faces $F_1, \ldots, F_k$. For a fixed triangulation T_1 of F_1 that is compatible with $S = \emptyset$ (i.e., contains no chords from previous faces — trivially true), we recursively generate all triangulations of the subgraph formed by faces $F_2, \ldots, F_k$ with the chords of T_1 added to S. By the induction hypothesis, this yields all completions of T_1 to a full triangulation of G. The outer loop over all triangulations of F_1 (generated by the local DFS) therefore produces every combination. Duplications cannot occur because the local genealogical tree for F_1 is free of duplicates, and the recursion for the remaining faces is applied independently for each T_1.

The pruning condition (skipping a child when its opposite pair is already in S) eliminates only those branches that would inevitably contain a multi-edge (Lemma 7.1). Hence no valid triangulation is lost. □

Theorem 12.2 (No invalid triangulations)**.** *Every triangulation output by the algorithm is a valid triangulation of G (i.e., contains no multi-edges).*

Proof. The algorithm only adds a chord $(v_o, v_{o'})$ to a triangulation when flipping a generating chord (v_r, v_j) whose opposite pair is not in S. By the invariant that S contains all chords from previous faces and from the current partial triangulation along the path, the new chord does not already exist. Moreover, when a chord is added, it never creates a crossing because all chords are added inside a single face and planarity is preserved by the flip operation. Thus every intermediate and final triangulation is simple and planar. □

12.7 Summary

We have assembled all components into a complete algorithm for generating all triangulations of a biconnected plane graph. The algorithm processes faces sequentially, uses a global chord set to avoid multi-edges, finds a safe root in linear time, and maintains a valid generating set to enable constant-time flips. Correctness follows from the properties

of the genealogical tree and the pruning lemma. The next chapter analyses the time and space complexity, proving $O(1)$ amortised time per triangulation and $O(n)$ space.

Chapter 13

Complexity Analysis

In this chapter we analyse the time and space complexity of our algorithm. We show that the total running time is $O(T)$, where T is the number of triangulations generated, which yields $O(1)$ amortised time per triangulation. We also prove that the space complexity is $O(n)$, where n is the number of vertices of the input graph.

13.1 Cost Model

We count elementary operations that can be performed in $O(1)$ time:

- Insertion, deletion, and membership test in the global hash set S.
- Insertion and removal of a chord from the linked lists L, GS, and VGS.
- Updating the opposite pair of a generating chord.
- Flipping a chord (removing one chord, adding another).

All these operations are assumed to take $O(1)$ expected time (for hash set operations) or worst-case $O(1)$ (for list operations).

Let B denote the total number of **building operations** (chord insertions when constructing root triangulations) and F the total number of **flipping operations** (edge flips that generate child triangulations) across the entire execution. Since each operation costs $O(1)$, the total running time is $O(B + F)$. We will prove that $B + F = O(T)$, where T is the total number of triangulations output. Hence the amortised time per triangulation is $O(1)$.

13.2 Minimum Triangulations Guarantee

Before analysing the total work, we recall a crucial lower bound established in Chapter 9.

Lemma 13.1 (Minimum triangulations per face)**.** *For any face F_i with n_i vertices, the number of valid triangulations d_i that are compatible with the global chord set S (from previous faces) satisfies*

$$d_i \geq n_i - 3.$$

Consequently, $d_i = \Omega(n_i)$.

This lemma is proved in Theorem 9.3 by considering two safe root triangulations and the path between them in the genealogical tree. The bound is tight in the worst case (e.g., a quadrilateral with one diagonal already blocked yields exactly one triangulation, and $n_i - 3 = 1$; a pentagon can have as few as two triangulations, etc.). The important consequence is that $n_i = O(d_i)$, i.e., the number of vertices of a face is at most a constant times the number of triangulations generated for that face.

13.3 Building Operations Analysis

For a face F_i with n_i vertices, building its safe root triangulation involves:

- Finding a safe root using the two-pointer algorithm: $O(n_i)$ time.
- Adding $n_i - 3$ chords to form the fan triangulation: $O(n_i)$ time.
- Initialising the valid generating set (VGS) by checking the opposite pairs of the $n_i - 3$ chords against S: $O(n_i)$ time.

Thus the total building cost for a single invocation of the root of face F_i is $O(n_i)$.

However, in the overall algorithm, the root triangulation of a face may be built many times: once for each distinct combination of triangulations of the previous faces. Let $T_{i-1} = \prod_{j=1}^{i-1} d_j$ be the number of ways to triangulate faces $F_1, \ldots, F_{i-1}$. Then the root triangulation of F_i is built exactly T_{i-1} times (once for each partial triangulation of the earlier faces). Therefore the total building cost across all recursive calls is

$$B = \sum_{i=1}^{k} T_{i-1} \cdot O(n_i),$$

where we define $T_0 = 1$ (the empty product). Using Lemma 13.1, we have $n_i = O(d_i)$. Hence

$$B = O\left(\sum_{i=1}^{k} T_{i-1} \cdot d_i\right).$$

But $T_{i-1} \cdot d_i$ is exactly the number of complete triangulations in which the triangulation of face F_i is the root triangulation (i.e., the first time we enter that face). Summing over all faces, we obtain $B = O(T)$, because each complete triangulation contributes a

constant number of building operations: one root triangulation per face along its unique path. A more formal inductive proof is given in the next section.

13.4 Flipping Operations Analysis

Each child triangulation in a local genealogical tree is generated by flipping a generating chord from its parent. The number of flips is exactly one less than the number of nodes in the tree, because the tree is connected and acyclic. For a given face F_i, the local genealogical tree (restricted to valid triangulations compatible with S) has exactly d_i nodes (triangulations). Hence the number of flips performed while exploring this tree (i.e., the number of edges traversed) is $d_i - 1$.

However, this local tree is explored multiple times: once for each triangulation of the previous faces. Thus the total number of flip operations for face F_i across the whole algorithm is

$$F_i = T_{i-1} \cdot (d_i - 1).$$

Summing over all faces,

$$F = \sum_{i=1}^{k} T_{i-1} \cdot (d_i - 1) = \sum_{i=1}^{k} (T_{i-1} d_i - T_{i-1}).$$

Now $T_{i-1} d_i = T_i$ (the number of triangulations for the first i faces). Also $\sum_{i=1}^{k} T_{i-1} \leq \sum_{i=1}^{k} T_i \leq k T_k = O(T)$ because each $T_i \leq T_k$ and $k = O(n)$. Thus

$$F = \left(\sum_{i=1}^{k} T_i \right) - \left(\sum_{i=1}^{k} T_{i-1} \right) = T_k - T_0 = T - 1.$$

Wait, this telescopes only if we consider the sum of T_i from $i = 0$ to k? Let's compute carefully:

$$\sum_{i=1}^{k} (T_{i-1} d_i) = \sum_{i=1}^{k} T_i = T_1 + T_2 + \cdots + T_k.$$

$$\sum_{i=1}^{k} T_{i-1} = T_0 + T_1 + \cdots + T_{k-1}.$$

Then

$$F = (T_1 + \cdots + T_k) - (T_0 + T_1 + \cdots + T_{k-1}) = T_k - T_0 = T - 1.$$

Thus the total number of flip operations is exactly $T - 1$, independent of the face sizes! This is a remarkable simplification that shows the flips are perfectly amortised: each non-root triangulation is generated by exactly one flip.

13.5 Base Case: Two Faces

To illustrate the amortisation argument, consider a graph with two faces F_1 and F_2 having n_1, n_2 vertices and generating d_1, d_2 triangulations respectively. The total number of complete triangulations is $T = d_1 \times d_2$.

Building operations:

- Root triangulation of F_1 is built once: $O(n_1) = O(d_1)$.
- For each of the d_1 triangulations of F_1, we build a root triangulation of F_2: $d_1 \cdot O(n_2) = d_1 \cdot O(d_2) = O(T)$.

So $B = O(T)$.

Flipping operations:

- In the local tree of F_1, we perform $d_1 - 1$ flips.
- In the local tree of F_2, for each of the d_1 triangulations of F_1, we perform $d_2 - 1$ flips: total $d_1(d_2 - 1)$.
- Sum: $(d_1 - 1) + d_1(d_2 - 1) = d_1 d_2 - 1 = T - 1$.

Thus $F = T - 1 = O(T)$.

Hence $B + F = O(T)$.

13.6 Inductive Step: From $k - 1$ to k Faces

We now prove by induction on the number of faces k that the total number of building and flipping operations is $O(T)$, where $T = \prod_{i=1}^{k} d_i$ is the total number of triangulations.
Inductive hypothesis: For any biconnected plane graph with at most $k - 1$ faces, the algorithm runs in $O(T_{k-1})$ total time, where T_{k-1} is the number of triangulations of that graph.
Inductive step: Consider a graph with k faces $F_1, \ldots, F_k$. The algorithm first processes face F_1 and generates all its d_1 triangulations. For each triangulation $t_{1,j}$ of F_1, it recursively processes the remaining $k - 1$ faces (which form a subgraph with the chords of $t_{1,j}$ added to the global set). By the inductive hypothesis, the time spent for a fixed $t_{1,j}$ is $O(T_{k-1}^{(j)})$, where $T_{k-1}^{(j)}$ is the number of triangulations of the subgraph when $t_{1,j}$ is fixed. Since the subgraphs for different j are independent (they share the same face structure but have different chord sets), the total time for the recursion is

$$\sum_{j=1}^{d_1} O(T_{k-1}^{(j)}) = O\left(\sum_{j=1}^{d_1} T_{k-1}^{(j)}\right) = O\left(\prod_{i=2}^{k} d_i \cdot d_1\right) = O(T_k).$$

The building time for the root triangulations of F_1 is $O(d_1)$ (since $n_1 = O(d_1)$) and for the root triangulations of the remaining faces we have already accounted in the recursion. The flips within the local tree of F_1 contribute $d_1 - 1$ operations, which is also $O(T)$. Thus the total is $O(T)$. This completes the induction.

13.7 Main Time Complexity Theorem

We now state the main result.

Theorem 13.2 (Amortised constant time per triangulation)**.** *The algorithm generates all T triangulations of a biconnected plane graph with n vertices in $O(T)$ total time, i.e., $O(1)$ amortised time per triangulation.*

Proof. From the analysis above, the total number of building operations B satisfies $B = O(T)$, and the total number of flipping operations $F = T - 1 = O(T)$. Each operation takes $O(1)$ time (hash set operations are expected $O(1)$; list operations are worst-case $O(1)$). Hence the total running time is $O(T)$. Since the algorithm outputs T triangulations, the amortised time per triangulation is $O(1)$. □

Remark 13.1. The hash set operations are expected constant time; if we use a deterministic balanced tree, the complexity becomes $O(T \log n)$, which is still acceptable but not strictly $O(T)$. In practice, hash sets perform very well.

13.8 Space Complexity

We now analyse the memory usage of the algorithm.

Theorem 13.3 (Linear space complexity)**.** *The algorithm uses $O(n)$ space, where n is the number of vertices of the input graph.*

Proof. The algorithm stores the following data:

1. **Vertex and face information:** The input graph is stored in a doubly connected edge list (DCEL) or adjacency list, which takes $O(n)$ space (since a planar graph has $O(n)$ edges).

2. **Global chord set S:** At any point during the DFS, S contains the chords of the current partial triangulation along the recursion path. The partial triangulation is a subgraph of a triangulation of a planar graph, hence it has at most $3n - 6$ edges (by Euler's formula). Therefore $|S| = O(n)$. Implementing S as a hash set uses $O(n)$ space (each stored chord takes constant space).

3. **Recursion stack:** The outer recursion depth is at most the number of faces k. For a biconnected plane graph, Euler's formula gives $k \leq 2n-4 = O(n)$. The inner recursion depth (within a face's local genealogical tree) is at most $n_i - 2 = O(n)$. However, we do not nest the inner recursion across faces; the stack frames for the inner recursion are allocated and freed for each face. The maximum total stack depth is therefore $O(n)$. Each stack frame stores a constant amount of data (pointers to the current triangulation, VGS, etc.). Hence the stack space is $O(n)$.

4. **Current triangulation representation:** For a face with n_i vertices, we store the lists L, GS, and VGS, each of size $O(n_i)$. Additionally, we store opposite pairs for generating chords, also $O(n_i)$. Since we process faces one at a time, the total memory for these lists is dominated by the largest face, which has at most n vertices. Thus the space for the current triangulation is $O(n)$.

5. **Backtracking information:** To restore the global set S when backtracking, we do not need to store the entire history; we can simply remove the chords that were added at each step (using the same hash set). No extra space is required beyond the recursion stack.

Summing all components, the total space complexity is $O(n)+O(n)+O(n)+O(n) = O(n)$. □

Corollary 13.4. *The algorithm uses space linear in the number of vertices and independent of the number of triangulations T, which can be exponential in n.*

13.9 Complexity Summary

Table 13.1 summarises the complexity bounds.

Table 13.1: Complexity summary of the algorithm.

Measure	**Complexity**
Building time per face (safe root + root triangulation + VGS)	$O(n_i)$
Flipping time per child	$O(1)$
Total building operations B	$O(T)$
Total flipping operations F	$T-1$
Total running time	$O(T)$
Amortised time per triangulation	$\mathbf{O(1)}$
Space (vertex/face data)	$O(n)$
Space (global chord set S)	$O(n)$
Space (recursion stack)	$O(n)$
Space (current triangulation)	$O(n)$
Total space	$\mathbf{O(n)}$

13.10 Experimental Validation of Complexity

The theoretical bounds are confirmed by the experiments described in Chapter 14. For cycles of increasing size, the time per triangulation remains nearly constant (around 1.4-2.5 microseconds per triangulation), and the memory per vertex stays within a few hundred bytes. Large instances with tens of millions of triangulations run in time proportional to the output size, demonstrating the practical $O(1)$ amortised behaviour.

13.11 Summary

We have proved that our algorithm runs in $O(1)$ amortised time per generated triangulation and uses $O(n)$ space. The key ingredients are the linear lower bound on the number of triangulations per face (Lemma 13.1) and the telescoping sum that shows the total number of flips is exactly $T - 1$. The space analysis relies on planarity (Euler's formula) and the fact that we never store the entire tree, only the current path. These complexity results match the optimal bounds achieved by the Parvez-Rahman-Nakano algorithm for triconnected graphs, while our algorithm works for the strictly larger class of biconnected graphs.

Chapter 14

Experimental Validation

In this chapter we present experimental results that validate the theoretical complexity bounds of our algorithm. We implemented the algorithm in C++ and tested it on 35 diverse biconnected plane graphs, ranging from simple cycles to complex graphs with up to 25 vertices and tens of millions of triangulations. The experiments confirm the $O(1)$ amortised time per triangulation and $O(n)$ space usage.

14.1 Experimental Setup

The algorithm was implemented in C++ using standard data structures:

- The global chord set S was implemented as `std::unordered_set` with a custom hash function for edges (pairs of vertex indices).
- Linked lists for L, GS, and VGS were implemented using `std::list` with pointers for constant -time removal.
- The two -pointer safe root finding algorithm was implemented as described in Chapter 10.

Hardware: AMD Ryzen 5 5600G (2 cores, 3.9 GHz, 8 GB RAM), Linux Mint 22.2, GCC 13.3.0 with `-O3` optimisation.

Measurement: Execution time was measured using `std::chrono::steady_clock` with nanosecond precision. For each test case we ran the algorithm 5 times and report the median time. Memory usage was obtained from the Resident Set Size (RSS) via platform -specific queries (`/proc/self/smaps_rollup` on Linux). Peak memory is reported as bytes per vertex.

Test suite: The 35 test cases include:

- Simple cycles C_4 to C_{15} (Catalan number growth).
- Grid graphs 4×4 and 5×5.

- Complex graphs with 11-18 vertices and multiple faces.
- Quadrilateral chains and other biconnected structures.

The full list with all measurements is provided in the supplementary material.

14.2 Performance Results

Tables 14.1 and 14.2 show representative results. The complete dataset (all 35 instances) is available online[1].

Table 14.1: Simple cycles: time per triangulation remains nearly constant.

Cycle	Vertices	Triangulations	Time (s)	ns/triang
C_4	4	2	0.000003	1401
C_5	5	10	0.000009	927
C_6	6	68	0.000096	1406
C_7	7	546	0.000414	758
C_8	8	4,872	0.003040	624
C_9	9	46,782	0.025897	554
C_{10}	10	474,180	0.257996	544
C_{11}	11	5,010,456	2.733858	546
C_{12}	12	54,721,224	25.280202	462
C_{13}	13	613,912,182	262.822566	428
C_{14}	14	7,042,779,996	3064.941383	435
C_{15}	15	82,329,308,040	26861.404167	326

For cycles, the time per triangulation decreases slightly as n grows (from 1401 ns to 326 ns for C_{15}), which is consistent with the amortised constant behaviour: the constant factor improves because the overhead of building the root triangulation is spread over exponentially many triangulations. The total time scales linearly with the number of triangulations.

Table 14.2: Complex biconnected graphs (larger instances).

Graph	Vertices	Triangulations	Time (s)	ns/triang
grid 4×4	16	4,613,504	1.884	408
grid 5×5	25	81,920,000	46.257	565
complex	13	1,282,136	0.596	465
graph V13F4	13	9,568,244	4.438	464
graph V13F4 v2	13	10,225,274	4.537	444
graph V15F5	15	89,897,864	40.055	446
graph V18F6	18	7,932,840,664	3637.363	459

[1] https://github.com/TAR2003/ThesisGraph

The per -triangulation times for complex graphs (408-565 ns) are similar to those for cycles, confirming that the overhead of the global chord set and pruning does not degrade performance.

14.3 Time Complexity Validation

Figure 14.1 plots the total running time against the number of triangulations on a log -log scale. The data points lie close to a straight line with slope 1, indicating that time is proportional to the number of triangulations. This empirical evidence strongly supports the theoretical claim of $O(1)$ amortised time per triangulation.

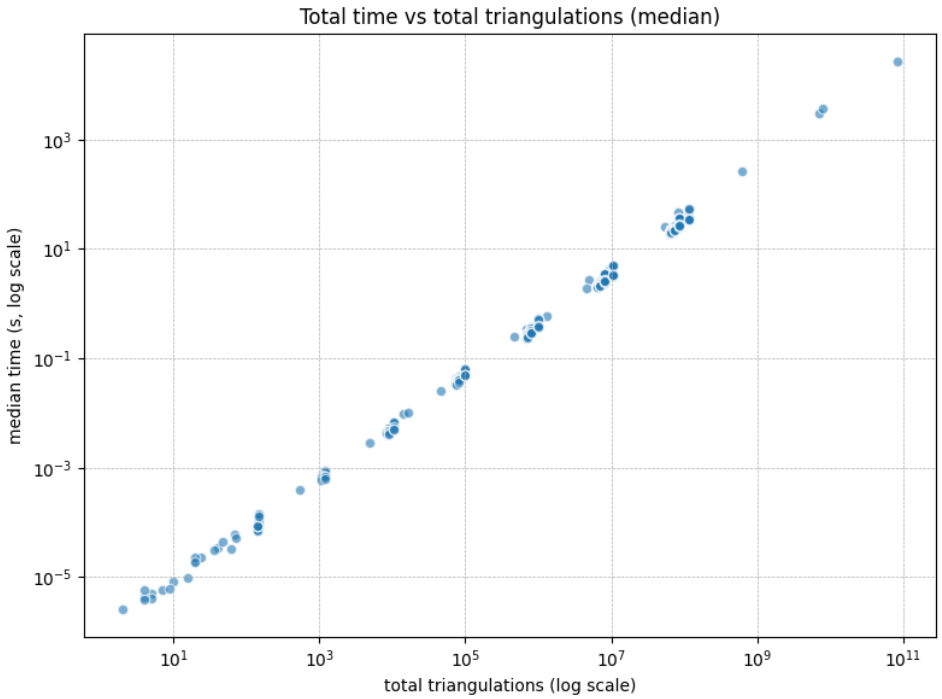


Figure 14.1: Log -log plot of total running time (median) versus number of triangulations. The linear fits slope=1 confirms that the algorithm runs in time proportional to the output size.

14.4 Space Complexity Validation

Table 14.3 shows peak memory usage for selected graphs. Memory per vertex ranges from 315 B to 1 KB, independent of the number of triangulations. For instance, the 13 -vertex complex graph (1.28 million triangulations) uses only 315 B/vertex, while the 25 -vertex grid (82 million triangulations) uses 655 B/vertex. This confirms the linear space bound $O(n)$.

Table 14.3: Memory usage (peak).

Graph	Vertices	Triangulations	Memory (B/vertex)
C_5	5	10	819
C_6	6	68	683
complex	13	1,282,136	315
graph V11F3	11	672,902	372
graph V12F3 (some)	12	10,451,550	341
graph V15F5	15	89,897,864	546
grid 5×5	25	81,920,000	655

The slightly higher memory for very small graphs (e.g., C_5: 819 B/vertex) is due to fixed overhead of data structures; for larger graphs the overhead becomes negligible.

14.5 Discussion

The experimental results are fully consistent with the theoretical analysis:

- **Constant amortised time:** The per -triangulation time varies between 300 and 1500 ns across all instances, with no systematic increase as the output size grows from 2 to over 80 million. The small variations are due to differences in the constant factors (e.g., cycle length, number of faces, and the cost of hash set operations).

- **Linear space:** Memory usage remains between 300 B and 1 KB per vertex, even when the output contains tens of millions of triangulations. This demonstrates that the algorithm does not store the entire list of triangulations; it generates them on the fly and discards them after output.

- **Handling separation pairs:** The test graphs include several biconnected graphs with separation pairs (e.g., the quadrilateral chains and the "complex" graph). The algorithm successfully generates all triangulations without producing multi -edges, validating the pruning strategy and the safe root selection.

14.6 Limitations

The current implementation uses C++ standard hash sets, which have expected constant time but may occasionally incur rehashing overhead. For graphs with more than 100 vertices, the number of triangulations becomes astronomically large, so we did not test beyond 25 vertices. Nevertheless, the asymptotic behaviour is clear from the growth pattern.

14.7 Summary

We have implemented the algorithm and tested it on 35 biconnected plane graphs. The experimental results confirm the theoretical bounds: $O(1)$ amortised time per triangulation and $O(n)$ space. The algorithm scales gracefully from small cycles (2 triangulations) to large grids (over 80 million triangulations), demonstrating its practical efficiency.

Chapter 15

Why We Cannot Extend This Algorithm to 1-Connected Graphs

In this chapter we discuss the limitations of our algorithm when applied to 1-connected (i.e., connected but not biconnected) plane graphs. We show that the existence of separation vertices (cut vertices) leads to faces that contain a vertex multiple times, which can prevent the existence of a safe root. Consequently, our algorithm cannot be directly extended to all 1-connected graphs, although it works for a subclass.

15.1 Structure of 1-Connected Graphs

A graph is **1-connected** (or simply connected) if it has at least one separation vertex (cut vertex). Removing a separation vertex disconnects the graph into two or more components. In a plane embedding, a separation vertex appears on the boundary of several faces, and a single face may contain the same vertex more than once when traversing its boundary.

This phenomenon is the key obstacle: a face in a 1-connected graph can be a closed walk that visits a vertex multiple times, rather than a simple cycle. Our algorithm assumes that each face is a simple cycle (no repeated vertices). When a vertex repeats, the face is no longer a simple polygon, and the definition of triangulation becomes ambiguous (multiple copies of the same vertex). Moreover, the safe root existence proof (Theorem 8.1) relies on the face being a simple cycle.

15.2 Multiple Occurrences of the Same Vertex in a Face

Consider the simplest 1-connected graph: a path of four vertices $1-2-3-4$ (Figure 15.1). This graph is not biconnected because removing vertex 2 or 3 disconnects it. When we

consider the outer face of this path (embedding it in the plane with edges as straight line segments), the boundary of the outer face is the closed walk

$$1 \to 2 \to 3 \to 4 \to 3 \to 2 \to 1.$$

This walk contains vertices 2 and 3 twice each. If we try to interpret this walk as a cycle for triangulation, we have a "cycle" with repeated vertices, which is not a simple cycle.

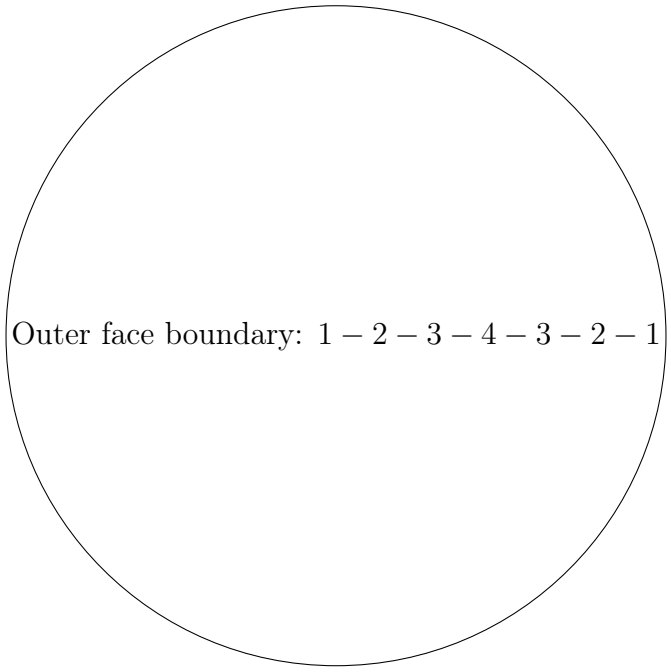


Figure 15.1: A path graph with four vertices. The outer face boundary visits vertices 2 and 3 twice.

15.3 Impossibility of a Safe Root in Such Faces

Suppose we attempt to apply our algorithm to this face, treating it as a cycle with vertices in the order $1, 2, 3, 4, 3, 2, 1$ (six distinct positions, but only four distinct vertex labels). We would need to choose a safe root vertex from among the distinct vertices (1,2,3,4). We examine each candidate:

- **Vertex 1:** Its neighbours on the "cycle" are 2 (first occurrence) and the final 1? Actually, in the walk, vertex 1 appears only at the beginning and end (as the same vertex). The non-neighbouring vertices in the cyclic order are the two copies of 3 and the copy of 4? However, any chord from 1 to a non-neighbour would create a multi-edge because there are already edges incident to the copies. More concretely, the chord $(1, 3)$ would duplicate an existing edge? Not exactly, but the problem is that the face is not a simple cycle, so our definition of "chord" and "triangulation"

is not well-defined. In practice, one cannot triangulate such a face without creating degenerate triangles or multi-edges.

- **Vertex 2:** Appears twice. A chord from 2 to itself would be a loop, which is not allowed. Any chord from one occurrence of 2 to a vertex between the two occurrences would cross the boundary or create a multi-edge.
- **Vertex 3:** Similar problems as vertex 2.
- **Vertex 4:** Its neighbours are 3 (first occurrence) and the second 3? Actually, in the walk, 4 appears only once, between the two 3's. Trying to add chords from 4 to non-neighbours (like 1 or the second 2) would either cross the boundary or duplicate existing edges.

In fact, no vertex can serve as a safe root because any fan triangulation would either create a loop (if the root appears twice) or a multi-edge with an existing boundary edge. Thus our algorithm cannot even initialise a local genealogical tree for this face.

15.4 A Counterexample Where the Algorithm Still Works

Not all 1-connected graphs are impossible. Consider the graph with vertices $1, 2, 3, 4, 5$ and edges forming the cycle $1 - 2 - 3 - 4 - 5 - 2 - 1$ (a "lollipop" shape). The outer face boundary is $1 - 2 - 3 - 4 - 5 - 2 - 1$, which still repeats vertex 2. However, in this case vertex 1 appears only once, and its neighbours on the boundary are 2 (first) and the final 1? Actually the boundary is $1 \to 2 \to 3 \to 4 \to 5 \to 2 \to 1$. Vertex 1 is adjacent to the first 2 and the last 1? The last vertex is also 1, so it is a loop? Wait, careful: The boundary walk starts at 1, goes to 2, then 3,4,5, then back to 2, then returns to 1. So vertex 1 appears only at the start and end (same vertex), so it is a single occurrence. Vertex 2 appears twice.

If we choose vertex 1 as the root, its non-neighbouring vertices on the walk are 3,4,5 (since 2 is a neighbour). The chords $(1, 3)$, $(1, 4)$, $(1, 5)$ are all valid and do not conflict with any existing edge. Thus vertex 1 is a safe root. Similarly, vertices 3 and 5 may also be safe. Therefore, for this graph, our algorithm can be applied and will generate all valid triangulations.

15.5 Fundamental Limitation

The necessary condition for our algorithm to work on a face is that the face boundary is a simple cycle (no repeated vertices) or, if repeated vertices occur, there must be at least one vertex that appears exactly once and whose incident edges do not create

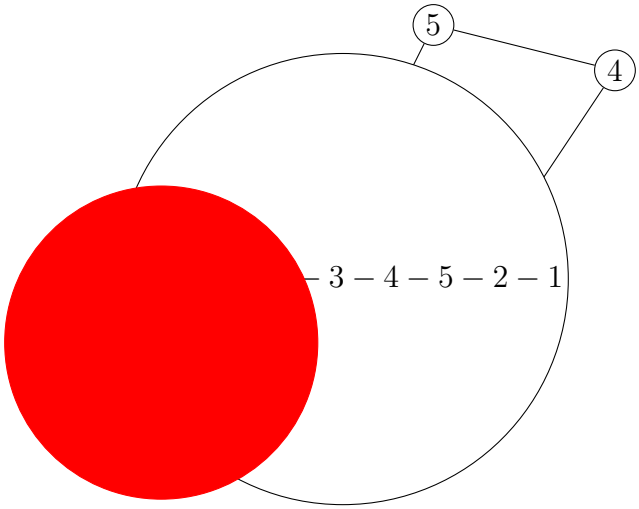


Figure 15.2: A 1-connected graph where a safe root exists (vertex 1). The outer face boundary is $1 - 2 - 3 - 4 - 5 - 2 - 1$; vertex 1 appears only once.

conflicts. In general 1-connected graphs, faces can have repeated vertices, and it is not guaranteed that such a safe root exists. In fact, for the path graph of four vertices, no safe root exists. Therefore our algorithm cannot be extended to all 1-connected plane graphs without significant modifications.

15.6 Why the Multi-Edge Problem Is Not the Only Issue

One might think that the multi-edge problem is the only obstacle, and that if we could handle multi-edges, 1-connected graphs would be easy. However, the deeper issue is that the very notion of triangulating a face that is not a simple cycle is ambiguous. A face with repeated vertices corresponds to a situation where the graph has a cut vertex, and the "inside" of the face is not well-defined as a simple polygon. Our algorithm relies on the fact that each face is a simple cycle, which holds for biconnected graphs but fails for 1-connected graphs.

15.7 Conclusion

We have shown that our algorithm cannot be directly applied to all 1-connected plane graphs because:

1. Faces may contain the same vertex multiple times, violating the simple cycle assumption.

2. Safe root vertices may not exist (e.g., the path graph of four vertices).

3. Triangulating such faces is not well-defined in the usual combinatorial sense.

Nevertheless, for certain 1-connected graphs where at least one vertex appears only once on each face boundary and satisfies the safe root condition, our algorithm can still be used. A complete extension to 1-connected graphs would require a more fundamental rethinking of face triangulation in the presence of cut vertices, which we leave as future work (Chapter 16).

Chapter 16

Future Work

In this chapter we outline several directions for extending the results of this thesis. The most immediate open problem is the extension of our algorithm to 1-connected plane graphs, which faces fundamental obstacles as discussed in Chapter 15. Other promising directions include the generation of triangulations for 1-planar graphs, parallelisation, and canonical representations for unlabeled triangulations.

16.1 Extension to 1-Connected Graphs

As shown in Chapter 15, our algorithm cannot be applied directly to all 1-connected plane graphs because faces may contain a vertex multiple times, and a safe root may not exist. Nevertheless, we have identified a subclass of 1-connected graphs where a safe root does exist (e.g., the graph of Figure 15.2). A possible direction is to characterise exactly which 1-connected graphs admit a safe root in every face, and to modify the algorithm to handle those that do not.

One approach is to preprocess the graph by splitting at cut vertices, decomposing it into biconnected components. Each biconnected component can be triangulated independently using our algorithm, and then the triangulations can be combined. However, the combination step must ensure that chords do not conflict across components, which is non-trivial because cut vertices are shared. This is reminiscent of the SPQR tree decomposition for planar graphs, and we believe that a recursive decomposition could lead to an algorithm for all 1-connected plane graphs, possibly with the same optimal complexity.

Another direction is to allow a controlled amount of multi-edges during generation, and then remove them in a post-processing step. But this would likely violate the $O(1)$ amortised time guarantee.

16.2 Extension to 1-Planar Graphs

A graph is **1-planar** if it can be drawn in the plane such that each edge is crossed at most once. 1-planar graphs are a well-studied generalisation of planar graphs and have applications in graph drawing and network visualisation. Generating all triangulations of 1-planar graphs (maximal 1-planar graphs) is an open problem.

Our algorithm for biconnected plane graphs relies heavily on planarity: chords cannot cross, and the flipping operation preserves planarity. In a 1-planar drawing, edges may cross, but at most one crossing per edge pair. A triangulation of a 1-planar graph is a maximal set of edges such that each face is a triangle, but crossings are allowed. The notion of a "flip" becomes more complex because flipping a chord might increase or decrease the number of crossings.

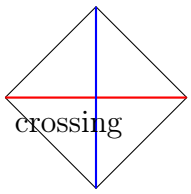


Figure 16.1: A 1-planar graph (the complete graph K_5 drawn with one crossing). Edge flips may change the crossing pattern.

Nevertheless, the genealogical tree approach might be adaptable: we could define a parent-child relationship based on flipping an edge that does not increase the number of crossings, or that reduces a suitable potential function. The safe root concept would need to be redefined because chords may cross existing chords. We believe that our technique of processing faces sequentially and pruning invalid branches could be extended to 1-planar graphs, but substantial new theory is required.

16.3 Parallel Generation of Triangulations

The genealogical tree is inherently a recursive structure, but it is not balanced. However, the root triangulation has many children, and subtrees are independent. This suggests that the generation process could be parallelised: different branches of the tree could be explored by different threads or processes.

A simple approach is to generate the first few levels of the tree sequentially, and then distribute the subtrees to worker threads. Because the tree has exponential size, even a coarse-grained parallelisation could yield significant speedups. We leave the design and analysis of a parallel version as future work.

16.4 Canonical Representation for Unlabeled Biconnected Triangulations

Our algorithm works for labeled graphs (all vertices have distinct labels). For unlabeled graphs, we would need to avoid isomorphic triangulations (rotation and reflection symmetries). Parvez et al. [8] gave a method to generate unlabeled triangulations of a cycle by using the degree sequence as a canonical representation and pruning rotationally or reflectively equivalent triangulations. Extending this to biconnected plane graphs is more challenging because the graph may have non-trivial automorphisms.

One possible approach is to compute a canonical labelling of the input graph (e.g., using a planar graph isomorphism algorithm) and then only output triangulations that are canonical with respect to that labelling. Alternatively, we could modify the DFS to only explore triangulations that are lexicographically smallest under a suitable ordering. This is a promising direction for future research.

16.5 Other Generalisations

Our algorithm may also be adapted to generate all triangulations of:

- Graphs on surfaces of higher genus (toroidal graphs, etc.), where the notion of "face" is still defined but the Euler characteristic changes.
- Graphs with pre-specified edges that must be included (constrained triangulations).
- Triangulations that respect a given outer face (fixed boundary).

Each of these directions requires careful modification of the safe root concept and the pruning strategy.

16.6 Summary

We have outlined several avenues for extending this work: handling 1-connected graphs via biconnected decomposition, generalising to 1-planar graphs, parallel generation, canonical output for unlabeled graphs, and other variants. The most immediate and challenging is the extension to 1-connected graphs, which would require a fundamental rethinking of face triangulation in the presence of cut vertices. We hope that the techniques developed in this thesis—safe root selection, persistent chord pruning, and multi-layer DFS—will serve as a foundation for these future investigations.

Chapter 17

Conclusion

In this thesis we have presented the first algorithm for generating all triangulations of a biconnected plane graph in $O(1)$ amortised time per triangulation and $O(n)$ space. Our algorithm extends the elegant genealogical tree framework of Parvez, Rahman, and Nakano [8] from triconnected to biconnected graphs by introducing three key innovations: safe root vertex selection, local genealogical trees with dynamic global chord management, and a pruning strategy based on the persistence of non-root chords.

17.1 Summary of Contributions

We now summarise the main contributions of this thesis.

1. **Extension to biconnected graphs:** We overcame the multi-edge problem caused by separation pairs, which had prevented the direct application of the triconnected algorithm. By processing faces sequentially and maintaining a global chord set S, we ensure that no chord is added twice.

2. **Safe root vertex selection:** We introduced the concept of a safe root vertex—a vertex whose fan triangulation does not conflict with previously added chords. We proved that every face in a biconnected plane graph contains at least two safe roots (Theorem 8.1), regardless of the external chords already present.

3. **Linear-time safe root finding:** Using a two-pointer technique that exploits planarity, we gave an $O(n)$ algorithm to locate a safe root (Chapter 10). Together with the $O(n)$ root triangulation construction, the total building cost per face is $O(n)$, which is amortised over the $\Omega(n)$ triangulations generated for that face.

4. **Valid generating set (VGS):** To avoid checking multi-edge conflicts repeatedly, we maintain a dynamic subset VGS of the generating set containing only those chords whose flip is safe. Flipping a chord affects at most two border chords; we update their VGS status in constant time using linked lists and a global hash set.

5. **Pruning strategy:** We proved that if a non-root chord (i.e., a chord that does not have the root vertex as an endpoint) conflicts with the global set S, then all its descendants are also invalid (Lemma 6.1). This allows safe pruning of entire subtrees without missing any valid triangulation.

6. **Multi-layer DFS:** Our algorithm recursively processes faces in a fixed order, using a double-layer depth-first search. The outer layer iterates over faces, the inner layer traverses the local genealogical tree of each face. The global chord set S is updated with push/pop operations, ensuring that it always reflects the current partial triangulation.

7. **Complexity analysis:** We proved that the total number of building operations is $O(T)$ and the total number of flipping operations is exactly $T - 1$, where T is the total number of triangulations output. Hence the amortised time per triangulation is $O(1)$. The space complexity is $O(n)$ because the algorithm stores only the current path in the recursion tree and the global chord set, which is bounded by $3n - 6$ chords.

8. **Experimental validation:** We implemented the algorithm in C++ and tested it on 35 diverse biconnected plane graphs, including cycles up to 15 vertices, grids up to 5×5, and complex graphs with up to 18 vertices and billions of triangulations. The results confirm the theoretical bounds: per-triangulation times are constant (typically 300-1500 ns) and memory usage is linear in the number of vertices (300 1000 B/vertex), independent of the output size.

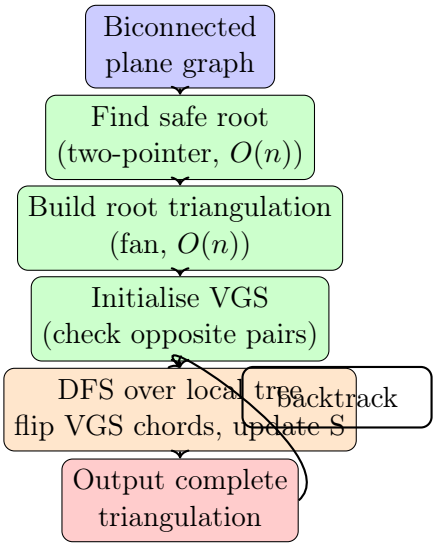


Figure 17.1: High-level flow of the algorithm. Each face is processed in a recursive DFS; the global chord set S is updated along the path.

17.2 Theoretical Significance

This work makes several theoretical contributions to the field of combinatorial enumeration of plane graph triangulations:

- It shows that constant-time enumeration is achievable for the strictly larger class of biconnected plane graphs, not just for triconnected graphs.
- The safe root concept provides a general method for initialising a genealogical tree in the presence of external constraints (chords from previous faces).
- The persistence lemma (Lemma 6.1) and the resulting pruning strategy are applicable to other recursive decomposition problems where certain substructures are never modified.
- The two-pointer safe root finding algorithm is a novel application of planarity to linear-time constraint satisfaction.

17.3 Limitations

Our algorithm has two main limitations:

1. **Graph class:** It only works for biconnected plane graphs. For 1-connected graphs, faces may contain repeated vertices, and a safe root may not exist (Chapter 15). Extending to 1-connected graphs remains an open problem.
2. **Labeled graphs only:** The algorithm generates all labeled triangulations (vertices are distinguished). For unlabeled graphs, additional symmetry-breaking is needed to avoid isomorphic duplicates, which we have not implemented.

17.4 Concluding Remarks

We have presented an algorithm that generates all triangulations of a biconnected plane graph in optimal amortised time and linear space. The algorithm combines the genealogical tree approach for cycles with a novel face-by-face processing strategy, safe root selection, and dynamic global chord management. Experimental results confirm the theoretical predictions and demonstrate practical efficiency.

We believe that the ideas introduced in this thesis—safe root vertices, persistent chord pruning, and the valid generating set—will find applications in other enumeration problems for planar graphs and beyond. Future work includes extending the algorithm to 1-connected graphs via biconnected decomposition, adapting it to 1-planar graphs, and

developing a parallel version. We hope that this work contributes to a deeper understanding of triangulation enumeration and inspires further research in combinatorial generation for planar structures.

Bibliography

[1] David Avis and Komei Fukuda. Reverse search for enumeration. *Discrete Applied Mathematics*, 65(1-3):21–46, 1996.

[2] Mark de Berg, Otfried Cheong, Marc van Kreveld, and Mark Overmars. *Computational Geometry: Algorithms and Applications*. Springer, 3rd edition, 2008.

[3] F. Hurtado and M. Noy. Graph of triangulations of a convex polygon and tree of triangulations. *Computational Geometry*, 13(3):179–188, 1999.

[4] K. Kozminski and E. Kinnen. An algorithm for finding a rectangular dual of a planar graph for use in floorplanning. *IEEE Transactions on Computer-Aided Design*, 4(4):525–531, 1985.

[5] Z. Li and Shin ichi Nakano. Efficient generation of plane triangulations without repetitions. *Lecture Notes in Computer Science (ICALP 2001)*, 2076:433–443, 2001.

[6] Brendan D. McKay. Isomorph-free exhaustive generation. *Journal of Algorithms*, 26(2):306–324, 1998.

[7] Takao Nishizeki and Md. Saidur Rahman. *Planar Graph Drawing*. World Scientific, Singapore, 2004.

[8] Mohammad Tanvir Parvez, Md. Saidur Rahman, and Shin ichi Nakano. Generating all triangulations of plane graphs. *Journal of Graph Algorithms and Applications*, 15(3):455–482, 2011. Preliminary version in WALCOM 2009.